

Solvita: Enhancing Large Language Models for Competitive Programming via Agentic Evolution

Han Li^{1*} Jinyu Tian^{3*} Rili Feng^{1*} Yuqiao Du^{2*} Chong Zheng² Chenyu Wang¹
Chenchen Liu³ Shihao Li¹ Xinping Lei¹ Yifan Yao¹ Weihao Xie¹ Letian Zhu¹
Jiaheng Liu^{1†}

¹Nanjing University ²Tsinghua University ³Independent Researcher

han.li.cs@smail.nju.edu.cn, liujiaheng@nju.edu.cn

Abstract

Large language models (LLMs) still struggle with the rigorous reasoning demands of hard competitive programming. While recent multi-agent frameworks attempt to bridge this reliability gap, they remain fundamentally stateless: they rely on static retrieval and discard the valuable problem-solving and debugging experience gained from previous tasks. To address this, we present Solvita, an agentic evolution framework that enables continuous learning without requiring weight updates to the underlying LLM. Solvita reorganizes problem-solving into a closed-loop system of strategy selection, program synthesis, certified supervision, and targeted hacking, executed by four specialized agents (Planner, Solver, Oracle, and Hacker). Crucially, each agent is paired with a trainable, graph-structured knowledge network. As the system operates, outcome signals—such as pass/fail verdicts, test certification quality, and adversarial vulnerabilities discovered by the Hacker—are recast as reinforcement learning updates to these network weights. This allows the agents to dynamically route future queries based on past successes and failures, effectively accumulating transferable reasoning experience over time. Evaluated across CodeContests, APPS, AetherCode, and live Codeforces rounds, Solvita establishes a new state-of-the-art among code-generation agents, outperforming existing multi-agent pipelines and nearly doubling the accuracy of single-pass baselines.

1 Introduction

Algorithmic problem-solving requires translating a natural language specification into a correct, efficient program. This demands precise formalization, deliberate strategy selection, and rigorous verification. Competitive programming benchmarks capture these demands in a controlled setting and have become a standard testbed for evaluating structured reasoning in large language models (LLMs) [1–4].

Despite rapid advancements, the dominant paradigm for LLM coding remains single-shot generation, which conflates understanding, planning, coding, and verification into one monolithic LLM call [5]. Recent multi-step pipelines, such as AlphaCodium [6] and MapCoder [7], mitigate this by introducing hierarchical planning and iterative refinement. Yet, they remain fundamentally stateless: each new problem is solved from scratch, and any experience gained from past mistakes is discarded. Retrieval-augmented generation (RAG) variants [8] attempt to add memory, but they treat retrieval as a static similarity lookup. Injecting raw text back into a prompt does not fundamentally alter the underlying reasoning procedure. In contrast, strong human programmers improve precisely because they accumulate transferable experience: they learn which strategies fit specific problem structures, recognize why certain implementations tend to fail, and learn to rigorously attack their own solutions before the judge does.

To bridge this gap, we introduce **Solvita**¹, a multi-agent framework that brings continuous, experience-driven evolution to LLMs for competitive programming. Solvita replaces static pipelines with a dynamic, closed-loop ecosystem of four specialized agents—a Planner, a Solver, an Oracle, and a Hacker. Rather than finetuning the massive parameters of the underlying LLM, Solvita pairs each agent with a trainable, graph-structured knowledge network. As the system solves problems and generates tests, it uses reinforcement learning to update the routing weights of these networks based on pass/fail verdicts and adversarial discoveries. Consequently, the framework accumulates and reuses experience across a stream

*Equal contribution.

†Corresponding author.

¹The name *Solvita* combines *solve* with the Latin *vita* (“life”; cf. Italian *vita*, English *vital*/*vitality*), evoking a solver that brings life to problem solving.

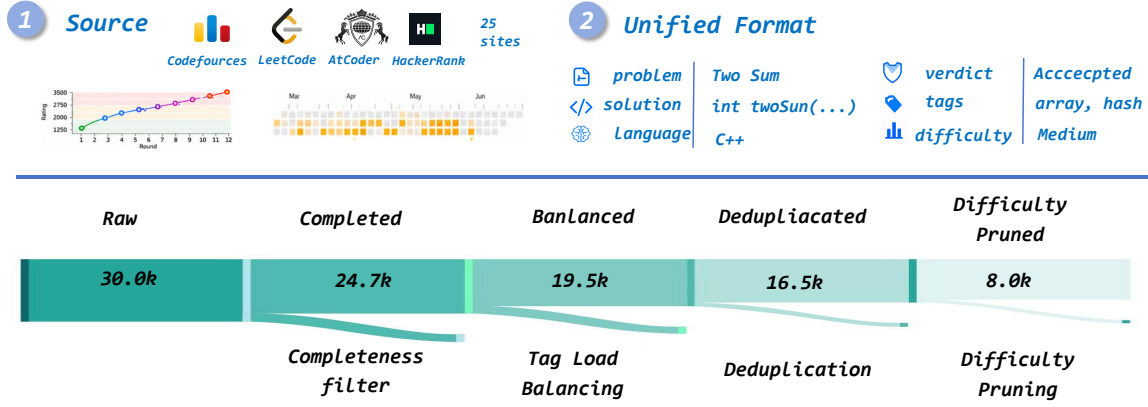


Figure 1: Data pipeline overview. Raw artifacts are collected from multiple competitive programming platforms, normalized into a unified JSON schema, and passed through four filtering stages: completeness, tag load balancing, embedding deduplication, and per tag difficulty pruning.

of tasks, allowing earlier solving and debugging episodes to directly shape how subsequent problems are approached.

(1) An agentic evolution framework: We reorganize problem-solving into a closed loop of strategy selection (Planner), program synthesis and patch-based repair (Solver), certified internal supervision (Oracle), and targeted adversarial testing (Hacker). This interactive loop ensures that failure signals propagate system-wide, allowing the agents to collaborate and cross-correct.

(2) Trainable knowledge networks as macro-level memory: Instead of a flat document store, each agent is backed by a structured graph linking problem queries, metacognitive analyses, and reusable algorithmic skills. Edge weights are dynamically adjusted by outcome signals rather than static semantic similarity. This transforms memory from passive retrieval into a learned routing mechanism that expands precisely where the agent currently struggles.

(3) Agentic feedback as a training signal: Solvita recasts the outcome signals produced by the Oracle and Hacker—such as certification quality and adversarial vulnerability events—as reinforcement learning signals. The LLM backbone remains entirely frozen, yet the system’s reasoning capability improves monotonically with use as the knowledge network learns to route problems to the correct strategies and failure-prevention tactics.

(4) Excellent performance: Solvita establishes a new state-of-the-art among code-generation frameworks on multiple benchmarks. Most notably, with a GPT-5.4 backbone on CodeContests, Solvita lifts pass@1 accuracy from 40.0% (single-pass) to 82.4%—nearly doubling the cold-start baseline while maintaining a similar token-consumption footprint to existing multi-agent pipelines.

2 Data

Each knowledge network requires a cold-start corpus before online learning can begin. We build this corpus in three steps—collection from heterogeneous competitive-programming sources, schema unification into a single JSON record format, and a four-stage filtering pipeline that controls completeness, tag balance, redundancy, and difficulty.

2.1 Collection

Our raw corpus is gathered directly from major competitive-programming judges—Codeforces, AtCoder, Aizu Online Judge, and a long tail of smaller platforms (e.g., LeetCode, SPOJ, UOJ)—rather than from any single pre-packaged dataset. Where applicable we cross-check against existing public collections such as CodeContests [1], CodeContests+ [9], and APPS [2] to recover editorial text and verdict labels that the raw scrape misses, but the unit of collection is the platform, not the dataset. The corpus is split into two subsets. The *training subset* keeps as much associated information as each source exposes, including statements, tests, metadata, editorials, and submissions with verdicts, and feeds the agentic-evolution pipeline. The *skill subset* pairs canonical reference problems with their official editorial solutions, seeding the downstream skill library.

2.2 Schema Unification

To reconcile the heterogeneous formats used across platforms, we normalize all collected artifacts into a unified JSON schema. Each record exposes a fixed set of canonical fields covering the problem statement, typed variable declarations and constraint bounds, sample and hidden tests stored as I/O pairs, editorial text, submission source with judge verdict and execution time, and algorithmic tags drawn from a controlled vocabulary. Interactive protocols, special judges, and multi-test-case packing conventions are mapped to standardized flags. The unified schema is the single input format consumed by all downstream stages.

2.3 Filtering

Starting from 30,018 problems, the unified corpus is refined through four sequential filters. The order is deliberate: tag load balancing precedes deduplication so that the expensive embedding-based dedup operates on a substantially smaller per-tag bucket, and difficulty pruning is applied last so that the floor is set against the post-dedup distribution rather than its inflated raw counterpart. ❶ **Completeness.** A problem is retained only if it carries both public and private test sets, algorithmic tags, a difficulty signal (e.g., rating, tier, or division), and a parseable I/O specification with explicit constraint bounds; this stage leaves 24,712 problems. ❷ **Tag load balancing.** A handful of tags would otherwise dominate the corpus and bias training toward their solution patterns, so we cap each tag at $C_{\max}$ problems and subsample the over represented tags, keeping the per tag distribution within a constant factor of the smallest tag and yielding 19,486 problems. ❸ **Deduplication.** We embed every statement with text-embedding-3-large, bucket by algorithmic tag, and compute pairwise cosine similarity within each bucket; any pair with similarity above δ is marked duplicate, and we retain the variant with more attached submissions and a larger test set, leaving 16,503 problems. ❹ **Difficulty pruning.** Trivially easy problems are removed because the base LLM solves them without experience augmentation: survivors are ranked by platform difficulty and any problem below the per tag floor $d_{\min}^{(\tau)}$ is dropped, producing the final corpus of 8,017 problems. The per platform difficulty mapping, the value of δ , the per tag floors $d_{\min}^{(\tau)}$, the cap $C_{\max}$, and the per platform raw and surviving counts are tabulated in Appendix A.

3 Solvita

3.1 Framework Overview

To this end, **Solvita** is built around four cooperating agents—a **Planner** that canonicalizes the problem and selects a paradigm, a **Solver** that implements the strategy and repairs it via search-and-replace patches rather than full regeneration, an **Oracle** that builds a certified internal test suite, and a **Hacker** that mounts adversarial attacks—coupled into one closed loop in which any failure signal propagates across all four agents (Figure 2). Since interaction signals vary widely in their informativeness, each agent is backed by its own trainable knowledge network under a role-specific schema. These networks share a common contextual-bandit policy [10], which surfaces the most informative precedents as advisory context at inference time and retires entries whose running reward falls below a deprecation threshold. Full policy details, featurization, and hyperparameters are deferred to Appendix B. Throughout the paper we use the single term *knowledge network* for these per-agent stores; the remainder of this section describes each agent in turn (Planner, Solver, Oracle, Hacker), together with the knowledge network and reward signal that trains it.

3.2 Planner

The Planner first reformulates the raw problem as a purely formal mathematical specification, stripping narrative framing and problem-irrelevant context to expose the underlying objective, variables, and constraints. From this canonical form it proposes a strategy: a predicted set of algorithmic tags, an implementation sketch, and a complexity estimate. On replanning after failure, the classified failure verdict guides revision. The exact `abstract_problem` prompt and JSON output schema are listed in Appendix E. The knowledge network behind the Planner stores strategy records linking the formalized problem to its predicted tags and downstream outcome, and at inference time the bandit policy of Section 3.1 retrieves precedents from structurally similar formalizations as planning advice.

Training. The Planner knowledge network is trained with the shared bandit update of Section 3.1 under a tag-prediction reward against the problem’s ground-truth tag set: each predicted tag earns $r = +1$ if it

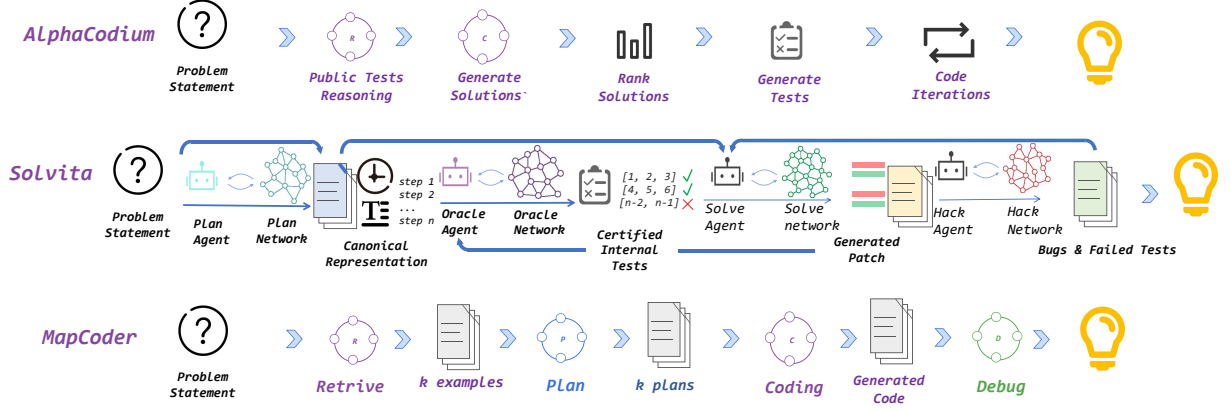


Figure 2: The Solvita architecture and its comparison with existing agent frameworks. Solvita couples an Oracle for certified internal-test construction, a Planner–Solver loop with patch-based refinement, and a Hacker that launches cascading adversarial attacks; failure signals propagate across all four agents’ knowledge networks (dashed arrows). In contrast to prior single-agent or pipeline-style code agents, Solvita closes the solve–certify–attack loop within one shared knowledge substrate.

matches a true tag and $r = -1$ otherwise, summed over the prediction. This directly teaches the network which formalized problem structures admit, which algorithmic paradigms.

3.3 Solver

The Solver implements the selected strategy as a C++ program, self-validating against public and Oracle-generated tests. On subsequent iterations, it applies *patch-based repair*: rather than regenerating the full solution, it emits search-and-replace edit blocks targeting only the diagnosed fault. A patch is accepted only if all regression tests (previously passing) still pass, preserving prior correctness while focusing effort on the unresolved case. The full prompt set (initial generation, patch decision, SEARCH/REPLACE patch, regeneration, and failure analysis) is given in Appendix E, and the storage layout, featurizer, and event-propagation mechanism for the Solver’s three-layer query–method–skill (QMS) knowledge network are documented in Appendix G.

The Solver is backed by a three-layer heterogeneous directed graph $\mathcal{G} = (\mathcal{V}_Q \cup \mathcal{V}_M \cup \mathcal{V}_S, \mathcal{E}_{QM} \cup \mathcal{E}_{MS})$ (Figure 3). **Q nodes** store the description and metadata of previously encountered problems. **M nodes** decompose solutions into function-block DAGs; *contrastive* M nodes pair a correct and incorrect solution sharing the same approach to isolate failure points, while *analysis* M nodes summarize accepted solutions as standalone trajectories. **S nodes** store annotated algorithmic skills with C++ templates, linked to M nodes via function-block identifiers (deterministic match or embedding fallback). Given a new problem q_{new} , the top- k similar Q nodes are retrieved and expanded into an activated subgraph; each reachable skill s receives a selection score aggregated over all two-hop paths,

$$\rho(s | q_{\text{new}}) = \sum_{\substack{q_i, m_j: \\ q_i \rightarrow m_j \rightarrow s \in \mathcal{G}}} \text{Sim}(q_{\text{new}}, q_i) \cdot w_{qm}^{(i,j)} \cdot w_{ms}^{(j,s)}, \quad (1)$$

where w_{qm} and w_{ms} are learned edge weights. Skills are sampled from $\pi(s) = \text{softmax}(\rho(s)/T)$ and assembled with their associated problem descriptions and contrastive analyses into a structured augmentation block.

Training via contrastive REINFORCE. The Solver’s training optimizes the Solver knowledge network without modifying the frozen LLM backbone. For each training problem, the agent solves it twice with the same backbone: once conditioned on the Solver knowledge network (full skill augmentation) and once without it (bare LLM). The outcome difference $\Delta R = R_{\text{with}} - R_{\text{without}}$ serves as a counterfactual reward isolating the network’s contribution, where R is the test pass rate. Edge weights are updated by REINFORCE:

$$\nabla_{\mathbf{w}} \mathcal{L} = -\Delta R \cdot \nabla_{\mathbf{w}} \log p(\mathbf{s} | q_{\text{new}}), \quad \mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} \mathcal{L}, \quad (2)$$

where the gradient decomposes through the chain rule from skill probabilities π through ρ to the underlying QM and MS edge weights, with MS weight groups renormalized after each update. The graph also grows dynamically: when both variants succeed, no node is added; when both fail, a new contrastive M node pairs the incorrect output with the closest correct solution from the corpus; when outcomes differ, the correct and incorrect outputs are directly paired. This ensures that the graph expands precisely where the agent currently struggles.

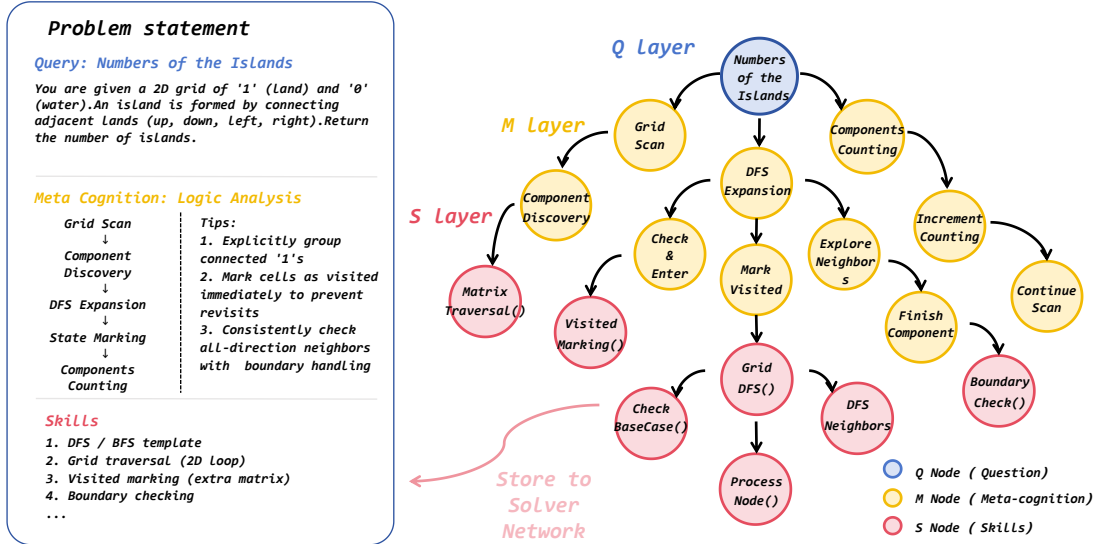


Figure 3: The three-layer Solver knowledge network. Q nodes (top) store problem descriptions and metadata; M nodes (middle) hold contrastive or analysis solution decompositions as function-block DAGs; S nodes (bottom) store annotated skills with code templates. Solid arrows denote deterministic links via function-block identifiers; dashed arrows denote semantic-similarity fallback. Edge thickness reflects learned weight magnitude.

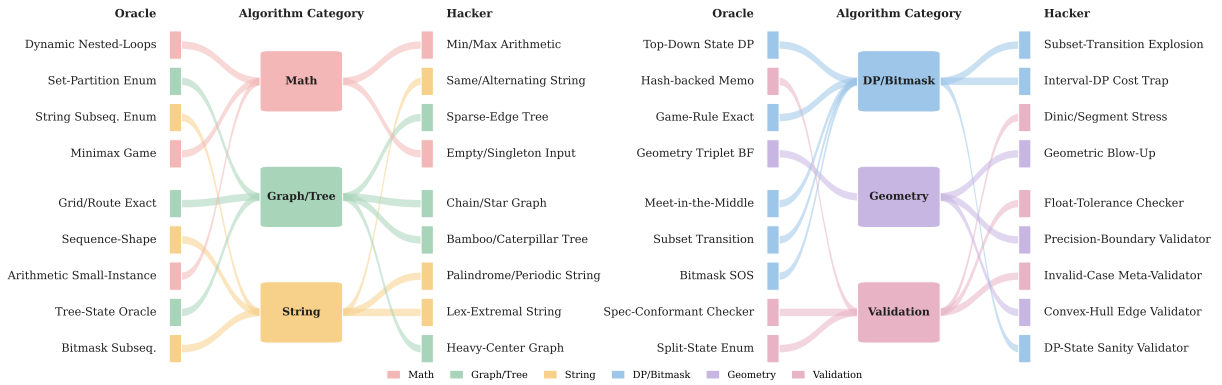


Figure 4: Seed-level strategy taxonomy of Oracle and Hacker memory, showing how each agent factorizes the shared algorithm space into its own reusable strategy units.

3.4 Category-Aligned Strategy Taxonomy of Oracle and Hacker Memory

Before specifying how the Oracle and Hacker are individually trained, we describe the shared structure their knowledge networks converge to and why that structure is functionally complementary. Figure 4 presents the seed-level view: the center column lists common competitive-programming categories, and the left and right columns show how Oracle and Hacker decompose that same space into different strategy families and representative reusable seeds.

The two sides overlap on categories but factor the same problem space in functionally different ways. Oracle strategies concentrate on routes that produce reliable supervision—DP/Search and Enumeration families dominate, with Decomposition and Domain-Aware as secondary modes—and align primarily with categories where reference solvers and cross-checkers are most informative (DP, Graph, Math, Bitmask, String). Hacker strategies, in contrast, concentrate on routes that expose latent bugs—Complexity/Worst-case and Structural/Topology dominate, with Boundary/Corner and Checker/Validation as secondary modes—and align with categories where stress testing and validator design carry the most signal (Stress, Checker, Graph, DP, String). Within each family the seeds are internally heterogeneous rather than collapsing to a single heuristic, with full counts and per-category percentages reported in Figure 4. This motivates Solvita’s design choice of *trainable* role-specific knowledge networks and the two distinct reward functions developed in Sections 3.5 and 3.6: instead of retrieving raw past examples, the system learns to route each new problem toward the most suitable certification strategy and the most informative adversarial-testing strategy.

3.5 Oracle

The Oracle produces certified supervision for the pipeline through four stages: (1) generate a testlib-based C++ generator, validator, and optional custom checker with iterative self-repair; (2) verify that the reference solver reproduces all public sample outputs; (3) generate N_{target} additional test inputs and certify each against an independent judge (custom checker > correct-solution runner > exact match), yielding a certification ratio $\rho = N_{\text{cert}}/N_{\text{target}} \in [0, 1]$; and (4) accept the artifact only after checking that the generated input set I and expected-output set O are nonempty, the certification ratio clears the threshold τ , and multi-answer routes provide a nonempty custom-checker evidence set C_{ma} :

$$A(x, f) = \mathbb{1}[|I| > 0 \wedge |O| > 0 \wedge \rho \geq \tau \wedge (\text{route}(x) \neq \text{multi_answer} \vee C_{\text{ma}} \neq \emptyset)]. \quad (3)$$

Artifacts failing the gate are discarded, and the Oracle retries with an alternative solver family. The Oracle is backed by a network of reference-solver strategy families $\mathcal{F}$ (e.g., top-down DP, constructive enumeration, brute-force verification), each annotated with applicable problem structures and historical success rates; the bandit policy of Section 3.1 selects the best-scoring family for each problem’s structural context. The four sub-prompts (generator, validator, checker, solver) and the stage-conditioned solver guidance appear in Appendix E.

Training. For each training problem x the Oracle picks a family $f \in \mathcal{F}$ and runs the four-stage pipeline, producing a certification ratio $\rho(x, f) \in [0, 1]$ and, when $\rho = 1$, an external judge verdict. The reward $r_{\text{oracle}}(x, f) \in [-1, +1]$ is the sum of (i) a partial-credit term proportional to ρ when $\rho < 1$, (ii) a full-certification bonus signed by the judge verdict when $\rho = 1$, and (iii) a failure penalty selected from four mutually exclusive failure modes (no failure / unready state / self-check failure / crash); the exact term coefficients, verdict scores, and penalty values are reported in Appendix C. The selected family is then updated by the bandit rule of Section 3.1 on the active feature keys $\Phi(x)$ for x (problem tags, constraint regime, paradigm class), so the resulting policy steadily concentrates on the family that yields the highest certified-test mass on each problem type.

3.6 Hacker

The Hacker searches for adversarial inputs that expose bugs surviving Oracle certification. A code analyst inspects the candidate via sandboxed execution and produces a structured vulnerability report (suspected bug class, attack hypothesis, recommended route). A cascading router then selects an attack route $u \in \mathcal{U} = \{\text{semantic}, \text{stress}, \text{antihash}\}$, instantiating respectively corner-case construction, maximum-bound stress testing, and lattice-based hash-collision generation. If the selected route fails, the system cascades through a fallback chain before declaring the candidate safe. The Hacker is backed by a vulnerability catalog recording exploitation types, triggering input patterns, successful attack routes, and algorithmic context; when a bug is discovered, the failure event propagates to all four knowledge networks, so each lesson is internalized once and reused everywhere. The Code-Analyst controller prompt and the three route-specific generator prompts (with their checklist/patch repair variants) are listed in Appendix E.

Training. For each candidate solution and chosen route u the Hacker runs one round, producing a verdict set V ; if no break is found, the cascade advances to the next route, up to a per-candidate budget of `max_hack_rounds` rounds (default 3, see Appendix F and the sensitivity sweep in Appendix H). Letting $V_{\text{valid}} \subseteq V$ be the inputs that pass the validator and $V_{\text{break}} \subseteq V_{\text{valid}}$ those that expose a failure, we summarize the round by three signals—the valid-input rate $g_{\text{valid}} = |V_{\text{valid}}|/|V|$, the break rate $g_{\text{break}} = |V_{\text{break}}|/\max(|V_{\text{valid}}|, 1)$, and an average severity g_{sev} over the broken inputs—and combine them, minus a compile-failure penalty, into

$$r_{\text{hack}}(x, u) = \text{clip}_{[-1, +1]}(w_v g_{\text{valid}} + w_b g_{\text{break}} + w_s g_{\text{sev}} - \kappa(c)). \quad (4)$$

The heaviest weight sits on actually breaking the candidate while still rewarding routes that produce valid, severity-graded inputs; the precise weights (w_v, w_b, w_s) , the per-verdict severity table $\omega(\cdot)$, the compile penalty $\kappa(c)$, and the degenerate-round correction for $|V_{\text{valid}}| = 0$ are listed in Appendix D. The selected route is updated by the same bandit rule as the Oracle on the active feature keys $\Phi(x)$, and any successful-break event additionally writes a contrastive entry into the Planner, Solver, and Oracle knowledge networks via the shared event bus, so the Hacker’s discoveries directly reshape the other three policies.

4 Experiments

We evaluate Solvita on three competitive-programming benchmarks — CodeContests [1] (CC, 165 problems), APPS [2] (1,000 sampled across tiers), and AetherCode [11] (AC, 400 problems) — and on recent Codeforces rounds. The main comparison uses five frontier backbones (GPT-5.4 [12], Claude Opus 4.6 [13], Qwen3.6, DeepSeek V4 Pro, Grok), with the same model powering every agent within a run; the more expensive diagnostic and component ablations report representative backbone subsets, always matched within each table. Pass@1 is the primary metric. We compare against commercial coding agents (Codex CLI, Claude Code), open-source agent frameworks (AlphaCodium [6], MapCoder [7], AgentCoder [14]), and a stateless single-pass generator. Decoding settings, pipeline budgets, and knowledge-network defaults are matched across methods; full configuration is in Appendix F and verbatim prompts in Appendix E.

4.1 Comparison with Code-Generation Agents

Solvita attains the best pass@1 in 14 of the 15 backbone–benchmark cells of Table 1, with the only exception on AetherCode under Claude Opus 4.6. The two commercial agents are each strongest on their home backbone but lose ground when moved to other models, while the open-source frameworks trail Solvita on every cell, with the gap widening on the harder AetherCode set. The lead grows on stronger backbones, indicating that knowledge accumulation and adversarial validation compound on top of an already capable solver rather than substituting for raw capability.

Table 1: Main results (pass@1, %) on CodeContests (CC), APPS, and AetherCode (AC). Each backbone is a column group; **bold** marks the best per column.

Method	GPT-5.4			Claude Opus 4.6			Qwen3.6			DeepSeek V4 Pro			Grok		
	CC	APPS	AC	CC	APPS	AC	CC	APPS	AC	CC	APPS	AC	CC	APPS	AC
Single-pass	40.00	37.90	18.00	44.85	40.10	22.75	33.94	30.80	9.50	47.27	42.50	24.00	38.18	34.60	15.50
Codex CLI	81.82	67.10	48.50	70.30	59.80	44.25	60.00	47.90	22.50	79.39	66.30	45.00	73.33	57.20	31.50
Claude Code	70.91	60.40	42.75	80.00	69.00	54.25	58.79	46.30	21.50	76.97	64.50	43.75	72.12	55.50	29.75
AlphaCodium	60.61	52.40	33.00	64.24	56.20	36.50	53.33	42.30	14.50	70.91	53.40	33.00	60.61	46.10	21.00
MapCoder	57.58	54.30	30.50	60.00	55.10	38.25	50.91	41.00	16.00	66.67	54.80	35.00	62.42	44.20	19.50
Solvita (ours)	82.42	67.70	49.25	80.61	69.30	53.75	69.70	55.10	26.00	89.09	68.10	51.50	78.18	58.50	33.50

Beyond pass@1, we also inspect the cost and failure profile of the main comparison. Figure 5 shows that Solvita stays in the same token-consumption band as open-source agent frameworks rather than matching the substantially higher footprint of commercial CLI agents. The residual-failure decomposition further shows that the gain is not concentrated in a single easy category: relative to the bare single-pass model, Solvita reduces algorithmic, specification-level, complexity, memory, and runtime failures across all three benchmarks.

4.2 Component and Knowledge-Network Ablation

Table 2 decomposes the contribution of each pipeline element. To establish clear baselines, we define **Single-pass** as a standard monolithic LLM generation without any multi-agent loop or persistent memory, and **without training** as the complete multi-agent framework running statelessly (i.e., with empty, untrained knowledge networks). These are compared against the **Full system**, where all four agents utilize their fully trained knowledge networks after the entire training trajectory. Switching from a single-pass generator to the multi-agent architecture without any persistent network already closes most of the gap to the full system, confirming the value of the closed-loop scaffold. Each trainable network is then probed at three checkpoints along the 5,318-problem training trajectory, taken at 1.5k, 3k, and 4.5k processed problems. Among the three knowledge networks, the Solver knowledge network gives the largest single-component gain on every benchmark, while the Hacker and Oracle knowledge networks add smaller but consistent margins; reading the same network’s three rows shows that gains genuinely accumulate with experience rather than appearing all at once. The full system surpasses any single-network addition on every backbone and at every checkpoint, so the components compound rather than substitute.

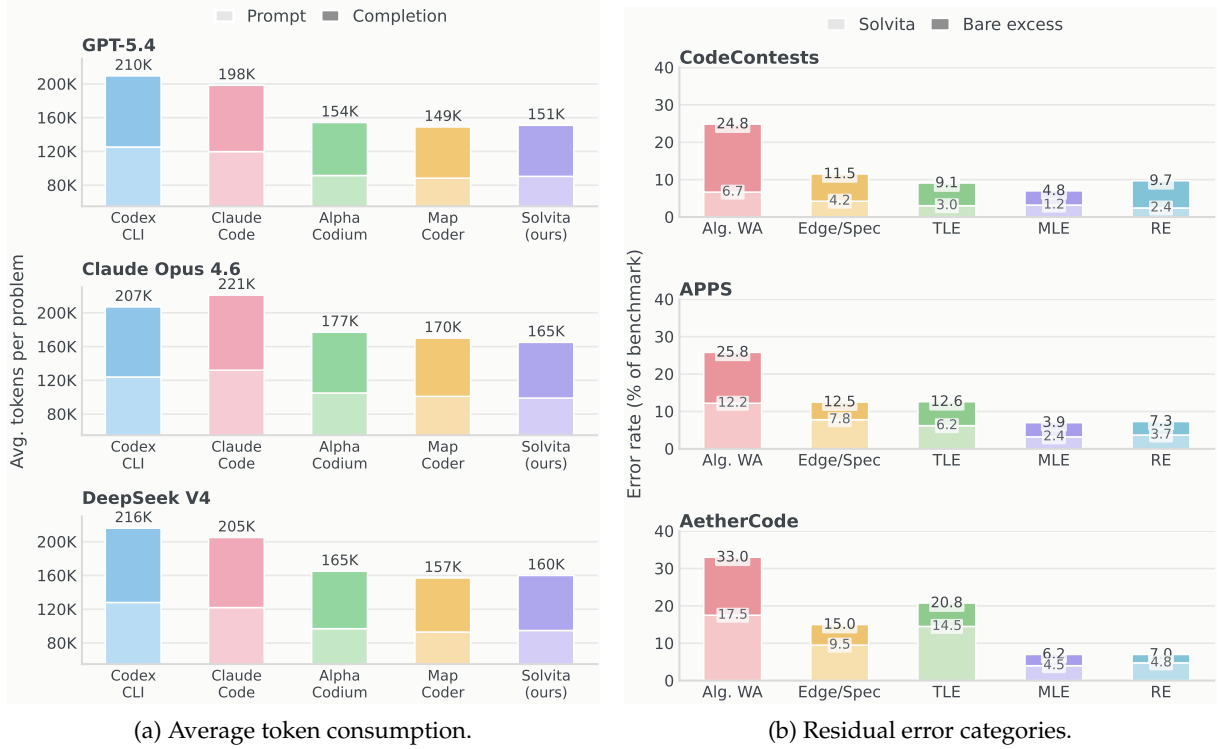


Figure 5: Cost and failure-profile analysis. (a) Average prompt and completion token consumption per problem, grouped by backbone and agent framework. Each bar stacks prompt and completion tokens. (b) Error-rate decomposition by benchmark and failure type under the representative GPT-5.4 backbone. The light segment shows Solvita’s residual error rate, and the darker segment (“Bare excess”) shows the additional error mass of the bare single-pass model; each bar top matches the GPT-5.4 bare-model error rate in Table 1. Error categories are defined as follows: Alg. WA (Algorithmic Wrong Answer), Edge/Spec (Edge-case or Specification mismatch), TLE (Time Limit Exceeded), MLE (Memory Limit Exceeded), and RE (Runtime Error).

Table 2: Additive ablation of pipeline components on CC, APPS, and AetherCode (pass@1, %). Each trainable network is reported at three checkpoints along the 5,318-problem training trajectory (@1.5k, @3k, @4.5k processed problems); static configurations are reported once. (Note: *Single-pass* is a monolithic baseline; *without training* is the stateless multi-agent framework; *Full system* includes fully trained networks).

Configuration	GPT-5.4			Claude Opus 4.6			Qwen3.6		
	CC	APPS	AC	CC	APPS	AC	CC	APPS	AC
Single-pass	40.00	37.90	18.00	44.85	40.10	22.75	33.94	30.80	9.50
without training	67.70	54.50	35.00	66.00	55.50	39.50	55.50	42.00	12.00
+ Solver network @1.5k	70.86	57.42	38.40	69.17	58.58	42.82	58.50	44.96	15.20
+ Solver network @3k	73.63	60.18	41.38	72.05	61.28	45.78	61.13	47.55	18.00
+ Solver network @4.5k	75.60	61.80	43.50	73.80	63.20	47.80	63.00	49.40	20.00
+ Hacker network @1.5k	69.42	55.90	36.40	67.80	56.82	40.90	57.10	43.44	13.40
+ Hacker network @3k	70.93	57.13	37.63	69.38	58.00	42.13	58.50	44.70	14.63
+ Hacker network @4.5k	72.00	58.00	38.50	70.50	58.80	43.00	59.50	45.60	15.50
+ Oracle network @1.5k	70.26	56.90	38.08	68.76	57.90	42.30	58.02	44.40	14.60
+ Oracle network @3k	72.50	58.95	40.78	71.18	60.00	44.75	60.18	46.50	16.88
+ Oracle network @4.5k	74.10	60.50	42.70	72.90	61.50	46.50	61.80	48.00	18.50
Full system	82.42	67.70	49.25	80.61	69.30	53.75	69.70	55.10	26.00

4.3 Patch-Based Repair vs. Full Regeneration

Table 3 compares the Solver’s patch-based inner loop against full regeneration under matched iteration budgets ($N_{\max} = 8$, identical retrieval, identical decoding). *Solve* is pass@1; *Iters* is the mean number of Solver iterations actually executed per problem (early-exit allowed once all tests pass); *TokSv* is the relative

completion-token saving against a fixed reference cost defined as “each iteration emits a full solution from scratch and the run consumes the entire iteration budget,” i.e. $T_{\text{ref}} = N_{\text{max}} \cdot \bar{t}_{\text{full}}$, where $\bar{t}_{\text{full}}$ is the average completion-token count of one full-regeneration draft on that benchmark; saving is then $1 - T_{\text{actual}}/T_{\text{ref}}$. Under this common reference, both strategies show some saving (full regeneration mostly through early-exit) and patch repair shows substantially more, because each post-draft iteration emits only a SEARCH/REPLACE block rather than a fresh solution. In this ablation, patch repair also attains a higher solve rate on every reported benchmark and backbone while running fewer iterations: regeneration plateaus several points lower because each retry rewrites the candidate from scratch and routinely breaks invariants the previous draft already satisfied. The remaining ablation protocol—LLM-skill selection and contrastive vs. non-contrastive Reinforce—is in Appendix H.

Table 3: Patch-based repair vs. full regeneration in the Solver inner loop, under matched iteration budgets ($N_{\text{max}} = 8$). *Solve* is pass@1 (%); *Iters* is mean Solver iterations per problem (early-exit allowed once all tests pass); *TokSv* is the relative completion-token saving against a fixed reference cost $T_{\text{ref}} = N_{\text{max}} \cdot \bar{t}_{\text{full}}$, where $\bar{t}_{\text{full}}$ is the average completion-token count of one full-regeneration draft on that benchmark, i.e. the cost of always running the full budget without early-exit and without patch reuse. Both strategies are scored against the same T_{ref} .

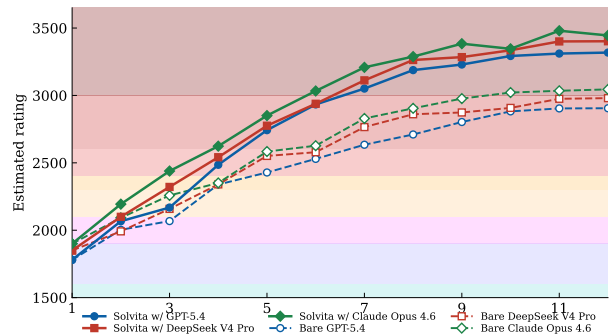
Backbone	Strategy	CodeContests (165)			APPS (1,000)			AetherCode (400)		
		Solve	Iters	TokSv	Solve	Iters	TokSv	Solve	Iters	TokSv
*GPT-5.4	Full regeneration	75.76	5.18	67.4	62.10	4.41	48.6	41.25	5.32	43.7
	Patch repair	82.42	3.74	91.2	67.70	3.16	88.5	49.25	3.95	86.3
*Claude Opus 4.6	Full regeneration	73.94	5.27	68.9	64.30	4.48	49.8	45.50	5.41	45.2
	Patch repair	80.61	3.86	92.0	69.30	3.24	89.4	53.75	4.03	87.5

4.4 Oracle and Hacker Diagnostic Contribution

Figure 6a isolates the two diagnostic modules across three backbones on a held-out set with known correctness labels, reporting wrong-solution detection, correct-solution preservation, and confirmed stronger-test rates. The Oracle alone is conservative and preserves correct solutions well, but misses subtle implementation bugs that only adversarial inputs expose. The Hacker alone detects more wrong solutions and surfaces more official-accept disagreements that survive accepted-solution cross-checking and manual validation. Combining the two gives the highest detection and stronger-test confirmation rates for every backbone while maintaining high preservation. The cross-backbone ordering follows the backbone strength in Figure 6b, but the Oracle–Hacker complementarity is stable across models. Metric definitions and the stronger-test confirmation protocol are given in Appendix H.

Backbone	Config.	Det’d	Pres’d	Str.
*Claude Opus 4.6	Oracle	82.4	96.8	8.5
	Hacker	87.6	92.9	11.4
	Both	92.8	96.0	19.6
*GPT-5.4	Oracle	79.8	96.1	7.4
	Hacker	85.2	91.8	10.2
	Both	90.6	95.4	17.9
*DeepSeek V4 Pro	Oracle	76.9	95.2	6.3
	Hacker	82.7	90.5	8.8
	Both	88.1	94.1	15.6

(a) Diagnostic quality of Oracle and Hacker.



(b) Codeforces rating-estimate progression across chronological contest rounds (x-axis).

Figure 6: Oracle/Hacker diagnostics and Codeforces evaluation across three backbones (Claude Opus 4.6, GPT-5.4, DeepSeek V4 Pro). **(a)** Det’d: wrong solutions detected; Pres’d: correct solutions preserved; Str.: Solvita-rejects/official-accepts disagreement problems confirmed as stronger-test cases. **(b)** Codeforces-style rating-estimate progression for the three Solvita backbones (solid) and the same backbones run bare (dashed), overlaid on the canonical Codeforces tier bands.

4.5 Codeforces Competition Evaluation

To complement offline benchmarks, we evaluate Solvita on recent Codeforces rounds (Div. 2 and Div. 1+2). Each contest is attempted in a single uninterrupted session within the official time limit, no corrections allowed after the window closes — the same constraints as human competitors. We use $K = 12$ post-cutoff contests (rounds 952–963, mixed Div. 2 and Div. 1+2), totalling 76 problems across A–F slots; contests were selected purely chronologically from the first 12 post-training-cutoff Codeforces rounds with publicly available official editorials, with no per-contest filtering. Figure 6b shows the Codeforces-style rating-estimate trajectory for the three backbones (GPT-5.4, DeepSeek V4 Pro, Claude Opus 4.6), computed by inserting each agent into the official standings and inverting the contest-local Elo expectation following CodeElo [15] and the classical Elo model [16]. All three Solvita variants converge into the Legendary Grandmaster band (≥ 3000) within $K = 12$ rounds, while the same three backbones run bare plateau in the high Grandmaster band (2700–2850), indicating that the gap into the Legendary Grandmaster range comes from the agentic loop rather than the underlying model. The three Solvita curves track each other within ± 80 rating points after round 6 (vs. a 140-point spread for the bare backbones), suggesting the gains transfer across backbones rather than depending on a particular model. The exact rank-insertion rule, rating inversion, aggregation, and contest-time configuration are given in Appendix F.5.

We evaluate Solvita on recent Codeforces rounds (Div. 2 and Div. 1+2). Each contest is attempted in a single uninterrupted session within the official time limit, no corrections allowed after the window closes — the same constraints as human competitors. Figure 6b shows the Codeforces-style rating-estimate trajectory computed by inserting each agent into the official standings and inverting the contest-local Elo expectation following CodeElo [15] and the classical Elo model [16]. All Solvita variants converge into the Legendary Grandmaster band (≥ 3000) within roughly a dozen rounds, while the same three backbones run bare plateau in the high Grandmaster band, indicating that the gap into the Legendary Grandmaster range comes from the agentic loop. The exact rank-insertion rule, rating inversion, aggregation, and contest-time configuration are given in Appendix F.5.

5 Related Work

Code generation and self-improving agents. LLM-based code generation has progressed from single-shot synthesis [1, 5] to structured multi-agent pipelines that add planning, retrieval, role separation, hierarchical decomposition, execution-based reranking, repository-level interfaces, and self-debugging or self-repair [6, 7, 14, 17–23], alongside general-purpose orchestrations and debate [24–28]; on top of this, self-improving agents update prompts, rationales, or pipelines through execution feedback or RL [29–36], but these methods either operate statelessly or improve communication and search rather than a persistent role-aligned memory. Solvita pairs role decomposition with per-agent knowledge networks coupled by adversarial feedback, optimizing a persistent graph by REINFORCE while the LLM stays frozen.

Memory and adversarial validation. Memory-augmented agents store and retrieve past experience through skill libraries, episodic reflection, virtual memory, or graph-structured reasoning [8, 37–43], but flat retrieval and the absence of role specialization remain reported bottlenecks. In parallel, systematic test generation spans fuzzing, equivalence modulo inputs, coverage-guided mutation, LLM-based fuzzing, certified competitive-programming validators, and dedicated hacking pipelines [9, 44–50]. Solvita differs in partitioning experience by agent role with learned edge weights, and in embedding certified test construction and adversarial attacks inside one closed-loop framework where each discovery updates all four knowledge networks at once.

6 Conclusion

We introduced Solvita, a framework that overcomes the limitations of stateless code generation by enabling continuous, experience-driven learning for frozen LLMs. By coupling four specialized agents (Planner, Solver, Oracle, Hacker) with dynamic, graph-structured knowledge networks, Solvita translates execution verdicts and adversarial testing into REINFORCE updates, allowing the system to accumulate algorithmic intuition, strategy routing, and debugging experience over time. Empirically, Solvita achieves a new state of the art across rigorous competitive-programming benchmarks—including live Codeforces rounds—nearly doubling the accuracy of single-pass baselines.

Limitations. Three trade-offs are visible in our runs. (i) Cold-start cost: the agentic loop is meaningfully more expensive than direct generation per problem, and the knowledge networks need on the order of 5,000 training problems before the per-problem cost is amortized into accuracy gains. (ii) Hacker scope:

anti-hash and lattice-based attacks are bounded by the backbone’s reasoning horizon, so heavily math-flavored failure modes (number-theoretic invariants, geometric tolerance bugs) remain under-covered. (iii) Patch-repair drift: on globally flawed candidates the Solver can mislabel a systemic flaw as localized and accumulate inconsistent edits before the iteration budget exhausts; the regression-rate signal in Section 4.3 catches this only post hoc.

Future work. Three directions follow naturally. First, warm-starting the knowledge networks from open-source experience corpora (editorials, accepted submissions, debugging traces) should shrink the cold-start window. Second, transferring the four-agent decomposition to other verifiable reasoning domains—formal theorem proving, where the Oracle becomes a proof checker and the Hacker searches for counter-models; mathematical olympiad problems, where certified test cases are replaced by symbolic verification; and scientific reasoning with executable simulators—is a direct port of the same closed-loop interface to settings that share competitive programming’s verifier-grounded reward structure. Third, the per-step adversarial signal produced by the Hacker is a candidate fine-tuning signal beyond prompt-level updates: investigating whether REINFORCE on knowledge-network weights can be lifted into model-weight updates without losing the role-aligned credit assignment is, to us, the most interesting open question.

References

- [1] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.
- [2] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *35th Conference on Neural Information Processing Systems (NeurIPS 2021), Track on Datasets and Benchmarks*, 2021.
- [3] Quan Shi, Michael Tang, Karthik Narasimhan, and Shunyu Yao. Can language models solve olympiad programming? *arXiv preprint arXiv:2404.10952*, 2024.
- [4] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida I. Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [6] Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with AlphaCodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500*, 2024.
- [7] Md Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. MapCoder: Multi-agent code generation for competitive problem solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [9] Zihan Wang, Siyao Liu, Yang Sun, Hongyan Li, and Kai Shen. CodeContests⁺: High-quality test case generation for competitive programming. *arXiv preprint arXiv:2506.05817*, 2025.
- [10] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, pages 661–670, 2010.
- [11] Zihan Wang, Jiaze Chen, Zhicheng Liu, Markus Mak, Yidi Du, Geonsik Moon, Luoqi Xu, Aaron Tua, Kunshuo Peng, Jiayi Lu, Mingfei Xia, Boqian Zou, et al. Aethercode: Evaluating llms’ ability to win in premier programming competitions, 2025. URL <https://arxiv.org/abs/2508.16402>.
- [12] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [13] Anthropic. The Claude 3 model family: Opus, Sonnet, Haiku, 2024. URL https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf. Anthropic Model Card.

-
- [14] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. Agent-Coder: Multi-agent code generation with effective testing and self-optimisation. *arXiv preprint arXiv:2312.13010*, 2024.
- [15] Shanghaoran Quan, Jiayi Yang, Bowen Yu, Bo Zheng, Dayiheng Liu, An Yang, Xuancheng Ren, Bofei Gao, Yibo Miao, Yunlong Feng, Zekun Wang, Jian Yang, Zeyu Cui, Yang Fan, Yichang Zhang, Binyuan Hui, and Junyang Lin. CodeElo: Benchmarking competition-level code generation of LLMs with human-comparable Elo ratings. *arXiv preprint arXiv:2501.01257*, 2025.
- [16] Arpad E. Elo. *The Rating of Chessplayers: Past and Present*. Arco Publishing, 1978.
- [17] Hung Le, Hailin Chen, Amrita Saha, Akash Gokul, Doyen Sahoo, and Shafiq Joty. CodeChain: Towards modular code generation through chain of self-revisions with representative sub-modules. In *International Conference on Learning Representations*, 2024.
- [18] Eric Zelikman, Qian Huang, Gabriel Poesia, Noah D. Goodman, and Nick Haber. Parsel: Algorithmic reasoning with language models by composing decompositions. In *Advances in Neural Information Processing Systems*, 2023.
- [19] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. Planning with large language models for code generation. In *International Conference on Learning Representations*, 2023.
- [20] Ansong Ni, Srinu Iyer, Dragomir Radev, Ves Stoyanov, Wen-tau Yih, Sida I Wang, and Xi Victoria Lin. LEVER: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning*, 2023.
- [21] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- [22] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations*, 2024.
- [23] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations*, 2024.
- [24] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- [25] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Awadallah, Ryan W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [26] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [27] Chen Qian, Yufan Dang, Jiahao Li, Wei Liu, Zihao Xie, Yifei Wang, Weize Chen, Cheng Yang, Xin Cong, Xiaoyin Che, Zhiyuan Liu, and Maosong Sun. Experiential co-learning of software-developing agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [28] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.
- [29] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C.H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 2022.
- [30] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, 2022.

-
- [31] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. In *Proceedings of the Conference on Language Modeling (COLM)*, 2024.
- [32] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023.
- [33] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [34] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.
- [35] Daya Guo et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [36] Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. A survey on self-evolution of large language models. *arXiv preprint arXiv:2404.14387*, 2024.
- [37] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [38] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *AAAI Conference on Artificial Intelligence*, 2024.
- [39] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [40] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [41] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.
- [42] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference (WWW)*, 2020.
- [43] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hofler. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024.
- [44] Vu Le, Mehrdad Afshari, and Zhendong Su. Compiler validation via equivalence modulo inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 216–226. ACM, 2014. doi: 10.1145/2594291.2594334.
- [45] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models. In *Proceedings of the 45th International Conference on Software Engineering*, 2023.
- [46] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023.
- [47] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. CodeT: Code generation with generated tests. In *International Conference on Learning Representations*, 2023.
- [48] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210*, 2023.

-
- [49] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50, 2024. doi: 10.1109/TSE.2023.3334955.
- [50] Jingwei Shi, Xinxiang Yin, Jing Huang, Jinman Zhao, and Shengyu Tao. CodeHacker: Automated test case generation for detecting vulnerabilities in competitive programming solutions. *arXiv preprint arXiv:2602.20213*, 2026.

Appendix

A Data Pipeline Configuration

This appendix lists the exact configuration of every step of the filtering pipeline in Section 2.3.

Per platform difficulty mapping. Each source platform exposes difficulty in a different unit; we normalize them into a single ordinal scale used by the difficulty signal in step (1) and the per tag floor in step (3). The mapping covers Codeforces rating bands, AtCoder score and color tier, LeetCode Easy/Medium/Hard, and the CodeContests difficulty tag.

Table 4: Per-platform difficulty normalization onto the Codeforces rating scale (800–3500). The same normalized tier is used across all platforms in step (1) and as the $d_{\min}$ cutoff in step (3). LeetCode labels map to a numeric range because they are coarse-grained; AtCoder scores and Codeforces ratings are used directly.

Platform	Native Difficulty Unit	Normalized Tier (CF Rating)	Typical $d_{\min}$ Cutoff
Codeforces	Problem rating (800–3500)	Same as native	800 / 900 per tag
AtCoder	ABC-A / ABC-B	800	800
	ABC-C	900–1100	900
	ABC-D	1200–1300	1200
	ABC-E	1400–1600	1400
	ABC-F	1700–1900	1700
	ABC-G / ARC / AGC	1900+	1900
LeetCode	Easy	800–900	800
	Medium	1000–1600	1000
	Hard	1500+	1400

Notes. (1) Codeforces ratings are on an 800–3500 integer scale derived from the Elo-based rating system. (2) AtCoder ABC mapping follows community-consensus ranges. (3) LeetCode mapping follows the widely cited rule that LeetCode difficulty roughly corresponds to Codeforces rating minus 600–700, with community-ratified bands. (4) The **Typical $d_{\min}$ Cutoff** column lists the per-tag difficulty floors commonly applied in step (3); the actual value is determined per tag at the 5th percentile of its post-deduplication difficulty distribution.

Tag load balancing. Step (2) caps each tag at $C_{\max} = 2300$ surviving problems. Tags above the cap are subsampled uniformly at random within their difficulty distribution so that the per tag count after balancing stays within a constant factor of the smallest surviving tag.

Embedding and deduplication threshold. Step (3) uses text-embedding-3-large with cosine similarity computed inside each tag bucket. The duplicate threshold is $\delta = 0.93$, chosen to maximize precision on a manually labeled validation set of 500 candidate pairs.

Retained tags and per tag difficulty floors. After steps (1)–(2) we retain $T = 107$ algorithmic tags. Each tag τ has its own difficulty floor $d_{\min}^{(\tau)}$ used in step (4), set so that the easiest surviving instance of τ is still non-trivial for the base LLM.

Per platform raw and surviving counts.

Table 5: Per-tag problem counts before and after tag load balancing (step 2). The fraction kept is computed as count after balancing divided by count before balancing.

Tag	Before Balancing	After Balancing	Fraction Kept
implementation	3819	2300	60.2%
math	3200	2300	71.9%
greedy	2840	2300	81.0%
dp	2625	2300	87.6%
data_structures	2329	2118	90.9%
constructive_algorithms	1759	1594	90.6%
brute_force	1429	1302	91.1%
graphs	1416	1359	96.0%
sortings	1409	1290	91.6%
binary_search	1403	1336	95.2%
dfs_and_similar	1357	1168	86.1%
trees	1225	1155	94.3%
strings	1154	1007	87.2%
number_theory	994	943	94.8%
combinatorics	804	788	98.0%

Notes. (1) Counts after balancing are computed after applying the per-tag cap $C_{\max} = 2300$. (2) Fractions are computed per tag as (after balancing) / (before balancing). (3) Statistics are computed over all tag occurrences; a problem carrying multiple tags contributes to each of its tags. The table lists the 15 most frequent tags in the corpus.

Table 6: Per-tag statistics and difficulty floors.

Tag	Retained after Step (3)	$d_{\min}^{(\tau)}$	Retained after Step (4)
implementation	1817	1400	890
math	1886	1600	719
greedy	1925	1600	951
dp	1891	1600	1503
data_structures	1783	1600	1366
constructive_algorithms	1322	1600	1109
brute_force	946	1400	782
graphs	974	1600	640
sortings	905	1400	653
binary_search	1003	1600	727
dfs_and_similar	957	1600	681
trees	806	1600	499
strings	745	1300	453
number_theory	684	1400	394
combinatorics	526	1400	375

Notes.(1) Statistics are computed over all tag occurrences; a problem carrying multiple tags contributes to each of its tags. The table lists the 15 most frequent tags in the corpus.

Table 7: Per-platform problem counts at each filtering stage. The total row sums to the corpus-wide figures reported in Section 2.3.

Platform	Raw Collected	After Completeness	After Balance	After Deduplication	After Difficulty Pruning
Codeforces	15862	15573	11634	8841	6325
AtCoder	4187	3984	3471	3316	756
Aizu	2645	2524	2276	2242	249
Others	7324	2631	2105	2104	687
Total	30,018	24,712	19,486	16,503	8,017

Notes. (1) The total row reproduces the corpus-level figures from the main text. (2) The final column reflects the corpus after applying the per-tag difficulty floors $d_{\min}^{(\tau)}$ (Table 6); a problem may be counted against its primary platform even if it was retained under different tags post-balancing.

B Contextual Bandit Policy Details

Each agent’s knowledge network uses the contextual bandit policy described in Section 3.1. The learning rate is $\alpha = 0.01$, rewards lie in $r \in [-1, 1]$, and a tag-overlap bonus of $+0.05$ per matching tag provides a prior. Feature keys encode the agent’s current finite-state-machine position (e.g., FSM: SOLVE_DRAFT), any failure type from the previous iteration (FAIL: TIMEOUT), and problem-level tags (TAG: dp). Parameters are persisted in JSON with atomic file-locking writes. Items whose running average reward drops below -0.3 after 20 or more uses are automatically deprecated.

C Oracle Reward: Failure-Path Details

A problem instance is $x = (d, c, p, \kappa)$ where d is the description, c the constraints, p the public samples, and κ the structural context. The Oracle produces a supervision artifact $y = (F, f^*, T, V, A, m)$ containing the candidate family set, selected family, certified tests, verifier provenance, acceptance indicator, and metadata. Family selection uses the bandit policy of Section 3.1,

$$f^*(x) = \arg \max_{f \in F(x)} s_f(x), \quad s_f(x) = b_f + \sum_{k \in \Phi(x)} W_{k,f}, \quad (5)$$

where $\Phi(x)$ is the set of active feature keys, and the acceptance gate follows Eq. 3. The successful-artifact reward defined in Section 3.5 is augmented with the following explicit negative penalties on failure paths, which are needed to keep the bandit signal informative when no certified test survives:

$$r_{\text{oracle}}^{\text{fail}} = \begin{cases} -1.0, & N_{\text{cert}} = 0 \text{ (crash or severe error),} \\ -0.7, & N_{\text{cert}} = 0 \text{ (self-check failure),} \\ -0.6, & \text{no valid tests or state not ready.} \end{cases} \quad (6)$$

For full certification ($\rho = 1$), the bonus $r_{\text{verify}} \in \{+1.0, -0.2, -0.5\}$ depends on whether the independent judge agrees, partially agrees, or contradicts the certified suite.

D Hacker Reward: Degenerate-Round Details

Let V be all sandbox verdicts, V_{valid} the valid-input subset, $V_{\text{break}} \subseteq V_{\text{valid}}$ those exposing a failure, and c the compilation failure count. The default per-round reward is the clipped composition of Eq. 4; on degenerate rounds (Gen_Failed with no valid verdict) the explicit correction $r = -0.6 - \min(0.3, 0.1 c)$ replaces the default so that repeated generator failures still produce a usable gradient. The judge is resolved in the same priority order as the Oracle: custom checker, correct-solution runner, exact match.

Component weights. The Hacker reward of Eq. 4 is a weighted combination of the valid-input rate g_{valid} , the break rate g_{break} , and the average severity g_{sev} , minus a compile-failure penalty $\kappa(c)$. The default weights and penalty are tabulated in Table 8. The bulk of the budget sits on w_b so that actually breaking the candidate dominates the gradient, while w_v keeps the router from collapsing to high-severity but invalid-by-validator inputs.

Quantity	Symbol	Value
Valid-input weight	w_v	0.20
Break-rate weight	w_b	0.55
Severity weight	w_s	0.25
Compile penalty (per failure, capped at 0.3)	$\kappa(c)$	$\min(0.3, 0.1 c)$
Reward clip range	—	$[-1.0, +1.0]$

Table 8: Hacker reward component weights and compile-failure penalty schedule.

Per-verdict severity. The severity table $\omega(\cdot)$ maps each per-input verdict to a scalar in $[0, 1]$ that feeds the round-level $g_{\text{sev}} = \frac{1}{|V_{\text{break}}|} \sum_{v \in V_{\text{break}}} \omega(v)$. Table 9 lists the values used across all reported runs; they encode the rough cost-to-fix ordering used by competitive-programming judges (a wrong-answer is cheap to triage, a sandbox crash is expensive).

Verdict	$\omega(\cdot)$
Wrong Answer (WA)	0.50
Time Limit Exceeded (TLE)	0.65
Memory Limit Exceeded (MLE)	0.75
Runtime Error (RE)	0.85
Sandbox crash / fatal signal	1.00

Table 9: Per-verdict severity weights used by the Hacker reward. Values were calibrated once on a 200-problem dev split and held fixed across all reported runs.

Degenerate-round sanity. When $|V_{\text{valid}}| = 0$ the round contains no informative break signal, but the generator may still have learned (or unlearned) something — e.g. producing inputs the validator rejects en masse. The fixed -0.6 baseline plus the linear compile penalty ensures the bandit signal stays bounded and nonzero so the route weights still update; without this correction, repeated `Gen_Failed` rounds would silently zero out the gradient and freeze the router on its current arm.

E Prompt Details

This appendix collects the actual prompt templates used by every agent in our experiments. All prompts are stored in a single YAML file (`config/prompt_template.yaml`) and rendered through a placeholder substitution layer (`<KEY>` tokens are filled at call time). For brevity we show the role-defining portions; cross-cutting boilerplate (output schemas, fast-I/O reminders, JSON-escaping warnings) is preserved verbatim where it materially affects behavior. The same prompts are used across all five backbones in our experiments.

Conventions. Unless noted otherwise, every JSON-output prompt below requires *strict* JSON: no Markdown fences, no commentary outside the object, and embedded newlines/tabs/backslashes escaped (`\n`, `\t`, `\\`). C++ outputs follow a single template: `#include-headers only (no <bits/stdc++.h>)`, C++17, fast I/O, with every major data-structure allocation justified against the input bounds; we explicitly highlight these only where the prompt diverges from this default. The TLE budget rule used throughout is the standard 10^8 ops/sec heuristic: `iterations / 10^8 ≤ time_limit_seconds`.

E.1 Planner

The Planner first reduces the raw problem statement to a canonical form and a tag set. Both the system message and the user message are short and JSON-bound so that downstream agents can parse the output deterministically.

Planner — abstract_problem (system)

You extract a precise canonical formulation and algorithmic tags for a competitive programming task. Respond with a single JSON object only. No markdown fences.

Planner — abstract_problem (user)

```
Original problem:
<PROBLEM_DESC>

Known problem types (hints): <PROBLEM_TYPES>

Constraints JSON:
<CONSTRAINTS_JSON>

Primary tags (level-1, coarse topics - prefer at least one when it fits):
<TAG_WHITELIST_LEVEL1>

Fine-grained tags (level-2 - optional; add when they precisely match the task):
<TAG_WHITELIST_LEVEL2>

Allowed tags reference (union of level-1 and level-2 vocabularies):
<TAG_WHITELIST>

Return JSON with this exact shape:
{
  "canonical_problem": {
    "objective": "string",
    "inputs": {},
```

```

"outputs": {},
"constraints": {"normalized": {}, "derived": []},
"required_properties": [],
"edge_cases": []
},
"algorithmic_tags_level1": ["primary tags from the level-1 list only"],
"algorithmic_tags_level2": ["optional fine-grained tags from the level-2 list only"],
"abstract_confidence": 0.0,
"abstract_trace": {"rationale": "why this abstraction fits", "warnings": []}
}
Rules:
- canonical_problem must be self-contained and precise.
- algorithmic_tags_level1 are the main coarse topic labels for this task; use only entries from the level-1 list above.
- algorithmic_tags_level2 are optional finer-grained labels; use only entries from the level-2 list above, and do not restate the same idea already covered by level-1.
- If a tag could fit both levels, put it in level-1 only and omit it from level-2.
- abstract_confidence in [0,1] reflects how well the statement constraints are satisfied.
- abstract_trace must briefly explain tagging and any ambiguity.

```

E.2 Solver: skill selection

After the canonical form is available, the Solver consults its three-layer QMS knowledge network (Section 3.3) and asks the LLM to pick a small number of skill identifiers and emit a sub-problem DAG. The strict JSON contract (no fences, exact id copying, no commentary) is enforced because the result is used as a direct lookup key into the skill table.

Solver — solver_skill_selection (system)

You output exactly one JSON object and nothing else.
Required keys: (1) "selected_skill_ids" (array of strings) – copy ids exactly from the candidate list;
(2) "subproblem_dag" (object) with "nodes" (array of {id, description}) and "edges" (array of {from_id, to_id}).
The DAG must be acyclic: edges mean dependency (from_id before to_id).
Use abstract algorithmic descriptions for nodes (not story fluff).
If no candidate skill applies, use [] for selected_skill_ids; you may still output a useful subproblem_dag for the **current** problem (abstract nodes/edges), or use empty arrays if no decomposition helps.
No markdown fences, no commentary, no text before or after the JSON.

Solver — solver_skill_selection (user)

You are selecting algorithmic **skills** (reference snippets) for a competitive programming task, and you must also output a **sub-problem DAG** that decomposes the **current** problem into abstract sub-goals.
Use the **problem summary**, the **activated graph context** (similar Q / M evidence), and the **candidate list** (ρ ranks retrieval relevance).
Choose **between** <MIN_SELECT> and <MAX_SELECT> (inclusive) skills that genuinely help solve the current task.
If **no** candidate is relevant, return an empty list for skills and empty DAG nodes/edges. Otherwise prefer **at least** <MIN_SELECT> skills when that many good fits exist in the list.

Problem summary

<PROBLEM_SUMMARY>

Activated graph context (top-Q similarity and linked M-nodes)

Structured evidence from the skill graph: similar problems (Q) and their logic pipelines (M).
Align skill choices with this retrieval context.

<ACTIVATED_GRAPH_CONTEXT>

Candidate skills (id, title, description, ρ)

<CANDIDATE_SKILLS_BLOCK>

Valid skill ids (copy exactly, one per line)

Your JSON strings must match one of these lines **verbatim** (including hyphens):

<VALID_SKILL_IDS_BLOCK>

Output format (strict)

Return **only** one JSON object and **nothing else** – no markdown fences, no commentary, no reasoning.

Required keys:

- "selected_skill_ids": string[] – ids copied verbatim from the candidate list.
- "subproblem_dag": { "nodes": [...], "edges": [...] }
 - nodes: each item is { "id": string (e.g. P1), "description": string } – abstract algorithmic sub-goal, not story text.
 - edges: each item is { "from_id": string, "to_id": string } – dependency (from_id must be solved before to_id). The graph must be a DAG (no cycles).

If selected_skill_ids is [], you may still output a meaningful subproblem_dag for this task, or use empty nodes/edges only when no decomposition is useful.

Solver — solver_skill_selection (user, minimal retry)

Output **only** one JSON object. No markdown fences, no reasoning text.
Include "selected_skill_ids" and "subproblem_dag" { "nodes": [...], "edges": [...] } (DAG, acyclic).
Choose <MIN_SELECT>-<MAX_SELECT> ids from the list below (or [] if none apply).

Task (short)

<PROBLEM_SUMMARY_SHORT>

Candidates (copy ids exactly from the bottom list)

<CANDIDATE_LINES_MINIMAL>

Valid skill ids (verbatim, one per line)

<VALID_SKILL_IDS_BLOCK>

Example:

```
{
  "selected_skill_ids": ["xxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"],
  "subproblem_dag": {
    "nodes": [{"id": "P1", "description": "Parse input and build data structure"}],
    "edges": []
  }
}
```

E.3 Solver: code generation and patch repair

The Solver alternates between full-program generation (initial draft) and search-and-replace patch repair (subsequent iterations, up to $N_{\max} = 8$). A separate patch_decision prompt routes between the two modes based on whether the failure looks localized or systemic. The generation prompt embeds an explicit *resource audit* requirement so that the model rejects sketches that would be infeasible at the stated constraint bounds.

Solver — generate_code.initial

You are solving a competitive programming problem. Apply the following three skills in order before writing any code.

Problem Context

Problem:

<PROBLEM_DESC>

<ABSTRACT_TAGS_LEVEL2_BLOCK>

Proposed algorithm:

<ALGORITHM>

Implementation steps:

<STEPS>

<CONSTRAINTS_BLOCK>

<PUBLIC_BLOCK>

<GEN_BLOCK>

<SOLVER_GRAPH_BLOCK>

<SELF_VALIDATION_BLOCK>

<MEMORY_ADVICE>

—

Skill 1: Brainstorming – Design Before Implementation

<HARD-GATE>

Do NOT write any C++ code until you have completed the design steps below.

This applies regardless of how simple the problem appears.

</HARD-GATE>

Produce a **concise** algorithm design (target: under 400 words total) covering:

1. **Core approach:** In 1–3 sentences, name the EXACT technique and data structures. Be specific: not "tree DP" but "rerooting DP storing subtree expected values"; not "greedy" but "leftist-heap merge tracking incremental cost per frontier". If you cannot name the specific data structure and key operation in one sentence, you have not converged – try a different angle before proceeding.
2. **Complexity + TLE check:** State time $O(?)$ and space $O(?)$. Estimate operations concretely. Apply the **10^8 simple ops/sec rule for C++**: if 'operations / $10^8 > \text{time_limit_seconds}$ ', the approach is TLE – redesign. Do not rationalize that constants

will save you.

3. **Correctness invariant:** The one-sentence key insight that makes this approach correct.

If the proposed algorithm in "Implementation steps" above would lead to an unsafe or hard-to-implement solution, you are free and encouraged to choose a better approach instead.

–

Skill 2: Test-Driven Development – Verify Algorithm Before Coding (Red Phase)

Iron Law: NO CODE WITHOUT A VERIFIED ALGORITHM.

Trace your chosen algorithm on each public sample input:

- Show key intermediate values and the final output your algorithm produces.
- Confirm it matches the expected output.

If any sample trace fails: **STOP. Redesign the algorithm. Do not write code for a failing approach.**

Only proceed to implementation after all samples pass the mental trace.

–

Skill 3: Writing Plans – Implementation Plan Before Coding

Before writing any code, decompose the implementation into bite-sized steps. For this problem, list:

1. **Data structures to define** (e.g., struct Node, priority_queue type, etc.)
2. **Main function steps** in order (e.g., "read input -> build trie -> run DP -> output")
3. **Any tricky sub-routine** that needs careful implementation (e.g., "merge two heaps", "BFS with state compression")

This plan is a 5–10 line outline, not prose. Write it down before touching code.

–

Skill 4: Implementation (Green Phase) + Verification Before Completion

Now implement following your plan. Requirements:

- Use standard C++ (C++17)
- Do NOT use non-standard headers like `#include <bits/stdc++.h>`
- Include all necessary standard headers explicitly
- Implement fast I/O
- Every major data structure allocation must be defensible given the maximum input bounds – state its size explicitly in a comment if non-obvious
- Prefer streaming, rolling-state, or sparse representations when full dense tables would exceed memory
- If the "Implementation steps" sketch above would lead to an unsafe implementation, adapt – do not follow it literally

After writing the code, before outputting it, verify:

- **Time (strict):** Count the dominant loop's iterations for maximum input. Use the **10⁸ ops/sec rule**: 'iterations / 10⁸ <= time_limit_seconds'. If you wrote $O(N * Q)$ and $N=Q=10^5$, that is 10^{10} ops = 100s – TLE. Redesign if this check fails. Do NOT assume constants will save you.
- **Memory:** For each major allocation, is the total within the memory limit?
- **Edge cases:** Does the code handle $n=1$ and $n=\max$?

If any check fails: fix the code first.

–

Output your full response in this exact structure:

Design

[Algorithm design, sample traces, and implementation plan from Skills 1–3 above]

Solution

“‘cpp

[Complete C++17 solution]

““

You are analyzing a competitive programming problem. Your task is ONLY to determine the best algorithm – do NOT write any C++ code in this response.

Discipline (non-negotiable)

TDD-style rule for competitive programming: Before declaring an algorithm correct, you must demonstrate it on actual computation, not mental simulation. The Iron Law:

> NO 'VERDICT: PROCEED' WITHOUT A '<run_cpp>' OR '<run_python>' BLOCK THAT EXECUTED AND PRODUCED THE EXPECTED OUTPUT IN THIS TURN.

"Mental traces" are forbidden as the sole evidence. If your reasoning concludes "and clearly the answer is X", you must run code that prints X.

Brainstorming rule (Codex 'brainstorming' skill): For any non-trivial problem, list 2-3 candidate algorithms with their complexity classes BEFORE picking one. State the trade-off (time vs. space vs. implementation difficulty) and pick the one that strictly fits the constraints. If only one approach fits, say so explicitly.

Verification rule (Codex 'verification-before-completion' skill): Evidence before claims. If you haven't run a verification command in this message, you cannot claim it passes. "It should work" is not evidence – 'stdout matches expected on tests T1, T2, T3' is evidence.

Anti-pattern: confidently wrong math. A formula that "looks right" can hide an algebraic step that breaks under specific inputs. The cure is brute force comparison on small random inputs, not more squinting at the formula. Two-parameter parameterizations of n-parameter structures are a classic trap; if your derivation reduced parameter count, write a brute force enumerator and confirm the count agrees on n=2, n=3, n=4 before trusting your reduction.

Anti-pattern: TLE that "should fit". A complexity that "should run in 0.5s" often runs in 8s due to constant factors (cache misses, branch mispredictions, integer division). When in doubt, compile a draft via '<run_cpp>' on a worst-case-shaped input near the constraint bound. If it doesn't finish in 5s in the sandbox, it won't finish on the judge.

–

Problem Context

Problem:

<PROBLEM_DESC>

<ABSTRACT_TAGS_LEVEL2_BLOCK>

Proposed algorithm hint (may be incomplete or wrong – use your own judgment):

<ALGORITHM>

<CONSTRAINTS_BLOCK>

<PUBLIC_BLOCK>

<ORACLE_STATUS_BLOCK>

<HARD_GATE_BLOCK>

<MEMORY_ADVICE>

–

Your Task: Algorithm Convergence (NO CODE)

You can verify your reasoning by embedding tool blocks in your reply. The harness executes them and feeds the output back to you:

– '<run_python>...</run_python>' – Python sandbox ('sys', 'math', 'itertools', 'collections', 'bisect', 'heapq', 'random'); 5s CPU. Use it to write a tiny brute-force solver and check small cases.

– '<run_cpp>...</run_cpp>' – Real C++ compilation + execution (full standard library; the same 'g++ -O2 -std=c++17' compiler the judge uses; 5s CPU). To pipe stdin, wrap it as:

```
““
<run_cpp>
INPUT_BEGIN
3
1 2 3
INPUT_END
#include <bits/stdc++.h>
using namespace std;
int main(){ ... }
</run_cpp>
““
```

The C++ tool is the BEST way to detect TLE early: pick a worst-case-shaped input near the constraint bound and run a candidate implementation. If it doesn't finish in 5s in the sandbox, it won't finish on the judge.

Strongly recommended workflow: write a Python brute force (small inputs, exponential is OK), write a candidate C++ implementation, run BOTH on the same small random inputs in tool blocks, and only declare PROCEED when their outputs agree on at least 5 random tests AND your C++ runs within budget on a worst-case-shaped input.

Think through the problem and converge on the best algorithm. Produce:

1. **Algorithm** (1-3 sentences, maximally specific): Name the exact data structure and core operation – e.g., "leftist-heap

merge tracking incremental cost per trie frontier node", not just "greedy" or "DP". If you are not certain, name the best candidate and explain why it fits.

2. **Complexity + TLE check:** State $O(\text{time})$ and $O(\text{space})$. Apply the 10^8 ops/sec rule: ' $\text{ops} / 10^8 \leq \text{time_limit_seconds}$ '. If this check fails, redesign before continuing.

3. **Correctness invariant:** One sentence stating why this approach produces the correct answer.

4. **Sample trace:** Trace your algorithm on the first public sample. Show key intermediate values and confirm the output matches.

5. **Implementation sketch** (3-5 bullets): The key data structures to define and the main function's control flow – specific enough that no further design decisions are needed when writing code.

6. **Verdict** – end your response with EXACTLY ONE of these two lines, on its own line:

- 'VERDICT: PROCEED' – if your algorithm clearly satisfies the time/memory limits and your derivation is sound.
- 'VERDICT: REDESIGN_NEEDED – <one-sentence reason>' – if your complexity analysis exceeds the time limit, your formula has a step you cannot verify, or you discovered an inconsistency. Be honest: choosing REDESIGN_NEEDED is preferred over forcing a flawed solution forward.

Stop here. Do NOT output any C++ code. The next step will ask for the implementation.

Solver — generate_code.code_only

Based on the algorithm design you provided above, implement the complete C++ solution.

Problem Context (for reference)

Problem:

<PROBLEM_DESC>

<CONSTRAINTS_BLOCK>

<PUBLIC_BLOCK>

<GEN_BLOCK>

<SELF_VALIDATION_BLOCK>

<MEMORY_ADVICE>

Implementation Requirements

Write the complete C++ implementation. Requirements:

- Use standard C++ (C++17)
- Do NOT use non-standard headers like `#include <bits/stdc++.h>`
- Include all necessary standard headers explicitly
- Implement fast I/O
- Every major data structure allocation must be defensible given the maximum input bounds – state its size in a comment if non-obvious
- Prefer streaming, rolling-state, or sparse representations when full dense tables would exceed memory
- If your algorithm sketch above would lead to an unsafe implementation, adapt – follow your own analysis, not the sketch literally

After writing the code, verify:

- **Time (strict):** Count dominant-loop iterations at max input. ' $\text{iterations} / 10^8 \leq \text{time_limit_seconds}$ '. If not, redesign.
- **Memory:** Each major allocation within memory limit?
- **Edge cases:** $n=1$ and $n=\text{max}$ handled?

Output your full response in this exact structure:

Verification

[Time/memory/edge-case checks]

Solution

“‘cpp

[Complete C++17 solution]

““

Solver — generate_code.patch_decision

You are choosing the repair mode for a failing C++ solution.

Problem:

<PROBLEM_DESC>

<ABSTRACT_TAGS_LEVEL2_BLOCK>

Algorithm:

<ALGORITHM>

```

Implementation steps:
<STEPS>

## Current Code:
```cpp
<PREV_CODE>
```

## Objective Failure Evidence:
<FAILURES_BLOCK>

<AGGREGATE_FAILURES_BLOCK>

<DIAGNOSTIC_BLOCK>

## Auxiliary References (use only as secondary hints):
<FEEDBACK_TEXT>

<FIXES_BLOCK>

<MEMORY_ADVICE>

## Your Task
Based on the objective evidence above:
1. First judge whether the failures indicate a localized bug or a systemic/global flaw.
2. Then identify the most likely error type.
3. Choose the better repair mode:
  - 'patch' if the overall approach is still sound and the issue is localized.
  - 'full_regen' if the overall approach is likely wrong or patching would keep drifting further.

Rules:
- Base the decision primarily on the objective evidence and current code.
- Treat the auxiliary references as secondary hints only.
- Do not guess beyond the provided evidence.
- Return ONLY valid JSON with this schema:
  {"mode": "patch|full_regen", "confidence": "low|medium|high", "reason": "one short sentence"}

```

Solver — generate_python_oracle

Write a Python brute-force oracle and a small-input generator. We will:

1. Run your brute force on each public sample input and verify it produces the expected sample output. If it doesn't, your understanding of the problem is wrong (or your brute force is buggy) and you'll be asked to revise.
2. Use your input generator + brute force as a reference to cross-validate the C++ solution we will write next on N random small inputs.

Both scripts must:

- Be syntactically valid Python.
- Use ONLY: 'sys', 'math', 'itertools', 'collections', 'bisect', 'heapq', 'random' (no other imports).
- Read input from stdin ('sys.stdin.read()') and write to stdout ('print' or 'sys.stdout.write').
- Run within 5 seconds CPU per invocation.

The brute force should be **maximally simple** — exponential time is fine because we only run it on small inputs. Resist the temptation to optimize; correctness for small N matters far more than speed.

The input generator must print exactly ONE valid input matching the problem's format, with **small** parameters (e.g. $n \leq 6$, values ≤ 10) so the brute force finishes quickly. Successive calls should produce different inputs.

Problem:

```

<PROBLEM_DESC>

<CONSTRAINTS_BLOCK>

<PUBLIC_BLOCK>

<FEEDBACK_BLOCK>

```

Output ONLY valid JSON with this schema (no markdown fence, no extra prose):

```

{
  "brute_force": "<full Python script>",
  "input_generator": "<full Python script>"
}

```

Solver — generate_code.patch (SEARCH/REPLACE format)

You are debugging a C++ solution that is FAILING tests. Your task is to generate SEARCH/REPLACE edits to fix the bugs.

Problem:

<PROBLEM_DESC>

<ABSTRACT_TAGS_LEVEL2_BLOCK>

Algorithm:

<ALGORITHM>

Implementation steps:

<STEPS>

Current Code (BUGGY):

“‘cpp

<PREV_CODE>

““

Test Failures:

<FAILURES_BLOCK>

<AGGREGATE_FAILURES_BLOCK>

<FEEDBACK_TEXT>

<FIXES_BLOCK>

<MEMORY_ADVICE>

Your Task:

Apply systematic debugging before proposing any fix.

Skill: Systematic Debugging – Root Cause Before Fix

Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST.

Phase 1 – Root cause investigation:

1. Read the failure evidence carefully. What does the wrong output tell you about where the code diverges?
2. Trace the failing test case through the code step by step. Track key variables at each step.
3. Identify the exact line or logic block where the value first goes wrong.
4. Name the root cause category: overflow / off-by-one / wrong formula / missing edge case / wrong data structure / TLE / MLE / other.

Phase 2 – Fix hypothesis:

- State one specific fix hypothesis that addresses the root cause.
- If the root cause is a global algorithmic flaw (not a local bug), this should be ‘full_regen’ territory – say so rather than patching.
- Re-check time and space complexity after the proposed fix.

Only after completing both phases, generate the SEARCH/REPLACE edits.

Every *SEARCH/REPLACE* edit must use this EXACT format:

«««< SEARCH

<exact contiguous code snippet from the current code>

=====

<replacement code with the fix>

»»»> REPLACE

CRITICAL RULES:

1. The SEARCH block must match the current code EXACTLY (including whitespace, indentation)
2. The SEARCH block must appear EXACTLY ONCE in the code
3. You can have multiple SEARCH/REPLACE blocks to fix multiple issues
4. Preserve proper indentation in the REPLACE block
5. Make minimal, surgical changes – only fix what’s broken
6. Re-check BOTH time and space complexity before proposing edits
7. Replace unsafe data structures if the current implementation appears to allocate memory proportional to a dangerous product of input dimensions
8. Do not preserve an existing approach just because it matches the plan if it is not implementable within the stated limits

Example:

«««< SEARCH

```
for (int i = 1; i <= n; i++) {  
    sum += arr[i];  
}
```

=====

```
for (int i = 0; i < n; i++) {  
    sum += arr[i];  
}
```

»»»> REPLACE

Generate the SEARCH/REPLACE edits now:

Solver — generate_code.regenerate

You are repairing a failing C++ solution. Apply the same design-first principles before rewriting.

Problem:

<PROBLEM_DESC>

<ABSTRACT_TAGS_LEVEL2_BLOCK>

Proposed algorithm:

<ALGORITHM>

Implementation steps:

<STEPS>

Previous Code (FAILING):

“‘cpp

<PREV_CODE>

““

<CONSTRAINTS_BLOCK>

<PUBLIC_BLOCK>

<GEN_BLOCK>

Test Failures:

<FAILURES_BLOCK>

<FEEDBACK_TEXT>

<FIXES_BLOCK>

<MEMORY_ADVICE>

—

Skill 1: Diagnose Before Rewriting

<HARD-GATE>

Do NOT rewrite code until you have diagnosed WHY the previous attempt failed and decided on a better approach.

</HARD-GATE>

1. **Root cause:** What specifically caused the failures? (wrong algorithm, wrong formula, wrong data structure, overflow, memory, TLE?)
2. **New approach:** If the algorithm itself is wrong, state the new correct approach with the same specificity required in initial generation. If only implementation was wrong, state what changes are needed.
3. **Complexity recheck:** Verify time $O(?)$ and space $O(?)$ for the new approach fit the constraints.

—

Skill 2: Sample Verification (Red Phase)

Before rewriting, trace the new approach on the failing sample cases to confirm it would produce the correct output.

—

Skill 3: Rewrite (Green Phase)

Requirements:

- Return ONLY the complete C++17 source code in a fenced block (no SEARCH/REPLACE).
- Keep valid parts of previous code when possible, but rewrite structure if the approach changed.
- Fix the listed failures and avoid regressions on passing cases.
- Every major allocation must be sized within bounds; state sizes in comments if non-obvious.

Output your full response in this structure:

Diagnosis and New Approach

[Root cause analysis and new algorithm design from Skills 1 and 2]

Solution

“‘cpp

[Complete C++17 solution]

““

Solver — generate_code.canonical_problem_block

Objective: <OBJECTIVE>

Inputs: <INPUTS_JSON>

Outputs: <OUTPUTS_JSON>

Constraints: <CONSTRAINTS_JSON>

Required Properties: <REQUIRED_PROPERTIES>

Solver — generate_code.self_validation_header

Self-validation failed: <FAIL_COUNT> issues in <TOTAL_RUN>/<TOTAL_VERIFY> cases tested:

Solver — generate_code.self_validation_footer

Please fix these issues.

E.4 Solver: failure analysis

When tests fail, the Solver invokes a structured failure-analysis prompt that forces the LLM to step-trace the simplest failing case before proposing fixes. The categorical error_pattern feeds back into the bandit's failure-type feature key (FAIL:TIMEOUT, FAIL:OFF_BY_ONE, etc., see Appendix B).

Solver — analyze_feedback.compilation

The following C++ code has compilation errors:

Code:
“`cpp
<CODE>
“`

Compilation Errors:
<ERROR_TEXT>

Provide:
1. Root cause of the errors
2. Specific fixes needed
3. Corrected code snippets

Be concise and actionable.

Solver — analyze_feedback.test_failure

You are a competitive programming debugging expert. Analyze the following failures and provide CONCRETE fixes.

Problem Description
<PROBLEM_DESC>

Selected Approach
Algorithm: <ALGORITHM>
Steps:
<STEPS_TEXT>

Current Status
Iteration: <ITERATION>
Pass Rate: <PASS_RATE>
Total Failed: <FAILED_COUNT>
Error Pattern: <ERROR_PATTERN>

Current Code
“`cpp
<CODE>
“`

Representative Failures (most important cases to fix)
<FAILURES_TEXT>
<DIAGNOSTIC_SECTION>

Your Task
1. Pick the SIMPLEST failure case above. Trace the code execution step-by-step with that input. Track key variables.
2. Identify WHERE and WHY the code produces wrong output.
3. Determine the root cause category: overflow, off-by-one, wrong formula, missing edge case, TLE, etc.
4. Provide SPECIFIC code-level fixes (not vague suggestions).

Return ONLY valid JSON (no markdown, no explanation outside JSON):

```
{
  "analysis": "<detailed step-by-step trace showing where the bug is>",
  "root_cause": "<one-line root cause>",
  "error_pattern": "<category: overflow/off-by-one/
wrong-formula/missing-edge-case/tle/other>",
  "suggested_fixes": [
    "<specific fix 1, e.g. 'Change line X: use long long instead of int'>",
    "<specific fix 2, e.g. 'Add special case handling when n==1'>"
  ]
}
```

Solver — analyze_feedback.hack_failure

You are a debugging expert. The solution passed all basic tests but FAILED adversarial hack tests.

Problem:
<PROBLEM_DESC>

Current Algorithm:
<ALGORITHM>

Implementation Steps:
<STEPS_JSON>

Code:
“‘cpp
<CODE>
““

HACK FAILURES (The code logic is likely correct for simple cases but fails edge cases):
<HACK_FAILURES_TEXT>

Task:
1. Analyze why the code fails these specific hack cases.
2. Identify the root cause (e.g. overflow, edge case, logic hole).
3. Provide a fixed C++ solution.

Return ONLY JSON:
{
 "analysis": "<analysis of hack failures>",
 "suggested_fixes": ["<fix 1>", "<fix 2>"]
}

E.5 Oracle: certified test generation

The Oracle prompt set has four sub-prompts (generator, validator, checker, solver) corresponding to the four artifacts that compose a certified test suite (Section 3.5). The generator and validator both use testlib as a strict scaffolding contract; the checker is asked to *independently verify* the candidate output rather than compare against a reference, with two distinct skeletons depending on whether the problem has a unique answer or admits multiple valid answers.

Oracle — generate_tests.generator

You are a generator agent. Write a C++17 program that outputs exactly one valid test case to stdout.

Hard requirements:

- Use testlib: #include "testlib.h"
- Do NOT use non-standard headers like #include <bits/stdc++.h>
- Call registerGen(argc, argv, 1)
- Use rnd.next(...) for randomness (no std::random, no srand/rand)
- Do not parse or set a random seed inside the program
- The harness will run your generator many times with different seeds and expects multiple distinct valid outputs
- Do NOT hardcode one fixed test case or ignore rnd.next(...)
- Different seeds should usually lead to meaningfully different inputs, not the same bytes every time
- Prefer generating from multiple structural families instead of a single canned example
- Do not print any extra text
- Prefer semantically targeted edge cases over generic large random data.
- When exact certification is important, favor modest-size cases that still expose tricky logic.

Minimal skeleton (illustrative only):

```
#include "testlib.h"
int main(int argc, char* argv[]) {
    registerGen(argc, argv, 1);
    return 0;
}
```

The program must produce ONE valid input instance that satisfies all constraints.

Problem Description:
<PROBLEM_DESC>

Constraints:
<CONSTRAINTS>

Public Tests:
<PUBLIC_TESTS>
<FEEDBACK_BLOCK>
<ADVICE_BLOCK>

Return ONLY a valid JSON object. No other text, no markdown, no code fences.

CRITICAL JSON RULE: All newlines inside string values MUST be escaped as \n. Tabs as \t.

Backslashes as \\. Do NOT include literal newline characters inside any JSON string value – this will break JSON parsing.

Schema:

```
{"generator_cpp": "#include ...\\nint main(){...}\\n"}
```

Oracle — generate_tests.validator

You are a validator agent. Write a C++17 program that reads one test case from stdin and validates it.

Hard requirements:

- Use testlib: #include "testlib.h"
- Do NOT use non-standard headers like #include <bits/stdc++.h>
- Call registerValidation(argc, argv)
- Validate the INPUT ONLY. Do NOT read any answer token, candidate output, or expected output.
- Stop immediately after consuming the valid input; then call inf.readEof()
- Use only these testlib input APIs unless strictly necessary: inf.readInt, inf.readLong, inf.readToken, inf.readSpace, inf.readEoln, inf.readEof, ensuref
- Do NOT use unsupported helpers such as peekChar or speculative stream-inspection APIs
- Use ensuref(...) to report specific constraint violations
- Include #include <unordered_set> if using unordered_set
- Exit 0 on success, non-zero on failure
- Do not print anything on success

Minimal skeleton (illustrative only):

```
#include "testlib.h"
#include <unordered_set>
int main(int argc, char* argv[]) {
    registerValidation(argc, argv);
    int n = inf.readInt();
    inf.readSpace();
    int m = inf.readInt();
    inf.readEoln();
    ensuref(n >= 1, "n must be >= 1");
    std::string s = inf.readToken();
    inf.readEof();
    return 0;
}
```

Problem Description:

<PROBLEM_DESC>

Constraints:

<CONSTRAINTS>

Public Tests:

<PUBLIC_TESTS>

<FEEDBACK_BLOCK>

Return ONLY a valid JSON object. No other text, no markdown, no code fences.

CRITICAL JSON RULE: All newlines inside string values MUST be escaped as \n. Tabs as \t.

Backslashes as \\. Do NOT include literal newline characters inside any JSON string value – this will break JSON parsing.

Schema:

```
{"validator_cpp": "#include ...\\nint main(){...}\\n"}
```

Oracle — generate_tests.checker

You are a checker/verifier agent. Write a C++17 program that **independently verifies** whether a candidate output is correct for a given input – WITHOUT needing any reference answer.

CRITICAL DESIGN PRINCIPLE:

Verifying a solution is far easier than computing it. Your checker must independently determine correctness by re-deriving the answer or checking invariants, NOT by comparing against a reference. DO NOT read from the ans stream at all.

Approach (choose one based on the problem):

- A) For problems with a unique answer: Compute the correct answer yourself using a simple brute-force algorithm (even $O(n^3)$ or $O(n^4)$ is fine – checkers run on small data), then compare against the candidate output.
- B) For problems with multiple valid answers: Read the candidate output and verify it satisfies all problem constraints (e.g., is it a valid permutation? Does the graph satisfy the required property? Is the value optimal?).

Hard requirements:

- Use testlib: #include "testlib.h"
- Do NOT use non-standard headers like #include <bits/stdc++.h>
- Call registerTestlibCmd(argc, argv)
- Read input via inf (the test input)
- Read candidate output via ouf (the output to verify)
- DO NOT read from ans – your checker must work without any reference answer
- Use quitf(_ok, "...") if the candidate output is correct
- Use quitf(_wa, "...") with a specific error message if incorrect
- Implement your own verification logic inside the checker
- Do NOT intentionally reject large valid inputs just because a tiny brute-force checker would be slow
- Never introduce empty objects unless the problem statement explicitly defines them

```
Minimal skeleton (approach A – unique answer):
#include "testlib.h"
#include <vector>
int main(int argc, char* argv[]) {
    registerTestlibCmd(argc, argv);
    int n = inf.readInt();
    int expected = brute_force_solve(n);
    int got = ouf.readInt();
    if (got != expected) quitf(_wa, "expected %d, got %d", expected, got);
    quitf(_ok, "ok");
}
```

```
Minimal skeleton (approach B – multiple valid answers):
#include "testlib.h"
#include <vector>
int main(int argc, char* argv[]) {
    registerTestlibCmd(argc, argv);
    int n = inf.readInt();
    int answer = ouf.readInt();
    if (!is_valid(answer)) quitf(_wa, "invalid answer");
    quitf(_ok, "ok");
}
```

Problem Description:
<PROBLEM_DESC>

Constraints:
<CONSTRAINTS>

Public Tests:
<PUBLIC_TESTS>
<CHECKER_ADVICE_BLOCK>
<FEEDBACK_BLOCK>

Return ONLY a valid JSON object. No other text, no markdown, no code fences.

CRITICAL JSON RULE: All newlines inside string values MUST be escaped as \n. Tabs as \t.

Backslashes as \\. Do NOT include literal newline characters inside any JSON string value – this will break JSON parsing.

Schema:

```
{"checker_cpp": "#include ...\\nint main(){...}\\n"}
```

Oracle — generate_tests.solver

You are an independent reference-solution author. Write a COMPLETE, COMPILABLE C++17 program that solves the following problem.

CRITICAL REQUIREMENTS:

1. Your code MUST be a complete standalone program with #include, main(), cin/cout.
2. Read input from stdin, write output to stdout, matching the exact I/O format shown in the public tests.
3. Correctness is the first priority. Prefer a semantically exact small-scale reference solver over a brittle formula.
4. The certification inputs are allowed to be modest; do NOT sacrifice correctness for aggressive large-constraint optimization.
5. The program MUST compile with: g++ -std=c++17 -O2
6. Keep the solver independent and trustworthy as a reference oracle.

Attempt guidance:
<STAGE_GUIDANCE>
<SOLVER_ADVICE_BLOCK>

Problem Description:
<PROBLEM_DESC>

Constraints:
<CONSTRAINTS>

Public Tests (your program MUST produce the exact expected output for these):
<PUBLIC_TESTS_BLOCK>

Algorithmic Strategy Reference (use for inspiration, do NOT copy verbatim):
<TEMPLATES_JSON>

<FEEDBACK_BLOCK>

Return ONLY a JSON object. No markdown, no explanation.

Schema:

```
{
  "selected_family_id": "<family_id from the strategy reference>",
  "template_name": "<name of the strategy you are using>",
  "solver_cpp": "<complete C++17 source code>"
}
```

Oracle — generate_tests.solver_stage (attempts 1 / 2 / 3+)

[Attempt 1] Write an independent reference solution. Prefer a straightforward polynomial-time algorithm, direct simulation, or simple derivation. Avoid factorial or exponential search unless the certification inputs are provably tiny. Prefer simple reference logic over contest-specific tricks.

[Attempt 2] The previous solver had correctness, runtime, or execution issues. Keep correctness first, but the solver must run within the certification limits. Do not use factorial or exponential search. Upgrade to a more scalable but still simple reference algorithm if needed.

[Attempt 3+] Write a robust reference solution that still remains easy to trust. It must run within the certification limits and address the observed failure mode. Do not use factorial or exponential search. If necessary, add sparse debug prints to stderr using `std::cerr << "TRACE: " << vars << std::endl;`. Never print inside tight loops unless sampled.

The Oracle also issues stage-conditioned guidance (attempt 1, attempt 2, attempt 3+) that biases the LLM toward progressively more scalable but still simple algorithms after each failed certification round; full reward shaping appears in Appendix C.

E.6 Hacker: code analyst and adversarial generators

The Hacker is implemented as a two-stage cascade. First, a *Code Analyst* controller inspects the candidate code, optionally calls sandboxed tools (`run_python`, `run_cpp`) to verify hypotheses, and emits a vulnerability report. Second, the report routes to one of three specialized generator prompts (*semantic*, *stress*, *anti_hash*). Each generator has its own *checklist* and *patch* prompts for SEARCH/REPLACE repair when its output is rejected by the validator.

Hacker — code_analyst.system

You are the Code Analyst controller for an adversarial hacker workflow.
Return ONLY valid JSON.
Do not include prose before or after the JSON.
If you need a tool, output a tool-call JSON object only.
If you are ready to conclude, output a final-report JSON object only.

Hacker — code_analyst.main

You are the Code Analyst, the strategy planner for an adversarial Hacker System.
Your goal is to find bugs, logic flaws, or vulnerabilities (WA, TLE, RE, MLE) in the provided C++ target code.

PROBLEM DESCRIPTION:
<PROBLEM_DESC>

CONSTRAINTS:
<CONSTRAINTS_JSON>

TARGET SOLUTION CODE (May contain bugs):
“‘cpp
<TARGET_CODE>
““
<ADVICE_SECTION>

AVAILABLE TOOLS:
You can verify your hypothesis by writing short probe codes. Do not guess blindly if you can test it!
Tool 1: ‘run_python’
Use this to perform precise mathematical calculations (e.g., combinations, large numbers) to check constraints and overflows.
Inputs: {"tool": "run_python", "parameters": {"script_code": "..."}}

Tool 2: ‘run_cpp’
Use this to compile and execute small C++ snippets to test specific runtime behaviors or replicate target logic.
Inputs: {"tool": "run_cpp", "parameters": {"cpp_code": "..."}}

RESPONSE FORMAT:
You must return a valid JSON object. You have two choices:
Choice A: Call a tool to gather information.
{
 "tool": "run_python",
 "parameters": {"script_code": "import math\nprint(math.comb(100, 50))"}
}

Choice B: Submit the Final Vulnerability Report (if you are confident).
{
 "bug_class": "overflow|hash_collision|index_oob|tle|logic_branch|unknown",
 "confidence": "high|medium|low",
 "evidence": ["e.g. math.comb(100,50) overflows 2^63"],
 "suggested_route": "anti_hash|semantic|stress",
 "input_hypothesis": ["large_n", "degenerate_tree", "collision_string"]
}

Note on routes: 'anti_hash' if polynomial hash is used. 'semantic' for most logical bugs. 'stress' ONLY if entirely clueless.

HISTORY OF YOUR ACTIONS & RESULTS:
<HISTORY_TEXT>

Analyze the code, call tools if needed to verify, and output valid JSON.

Hacker — code_analyst.json_repair

Your previous reply was not valid JSON for the Code Analyst protocol.

Rewrite the same intent as ONE valid JSON object.

Allowed outputs:

1. A tool call:

```
{
  "tool": "run_python|run_cpp",
  "parameters": {...}
}
```

2. A final report:

```
{
  "bug_class": "overflow|hash_collision|index_oob|tle|logic_branch|unknown",
  "confidence": "high|medium|low",
  "evidence": [...],
  "suggested_route": "anti_hash|semantic|stress",
  "input_hypothesis": [...]"
}
```

Previous reply:

<PREVIOUS_RESPONSE>

PROBLEM DESCRIPTION:

<PROBLEM_DESC>

CONSTRAINTS:

<CONSTRAINTS_JSON>

TARGET SOLUTION CODE:

“‘cpp

<TARGET_CODE>

““

<ADVICE_SECTION>

Return valid JSON only. Do not add any explanation, markdown, or commentary.

Hacker — code_analyst.force_tool

Your current vulnerability report is too weak to submit as a final answer.

You must call exactly one tool before you can submit a final report.

Do not submit a final report yet.

Return ONLY a tool call JSON object in one of these forms:

```
{
  "tool": "run_python",
  "parameters": {"script_code": "..."}
}
or
{
  "tool": "run_cpp",
  "parameters": {"cpp_code": "..."}
}
```

PROBLEM DESCRIPTION:

<PROBLEM_DESC>

CONSTRAINTS:

<CONSTRAINTS_JSON>

TARGET SOLUTION CODE:

“‘cpp

<TARGET_CODE>

““

<ADVICE_SECTION>

PREVIOUS WEAK REPORT:

<WEAK_REPORT_JSON>

HISTORY OF ACTIONS:

<HISTORY_TEXT>

Call one tool that will increase confidence in the bug class or input hypothesis.

Hacker — semantic.generator

You are the Semantic Generator, a specialized C++ coder for an adversarial Hacker System. Your job is to write a standalone C++ program that generates a single, highly-targeted test case designed to trigger the specific vulnerability described by the Code Analyst.

PROBLEM DESCRIPTION:

<PROBLEM_DESC>

<ADVICE_SECTION>

CONSTRAINTS (The output of your C++ generator MUST satisfy ALL of these):

<CONSTRAINTS_TEXT>

<PREVIOUS_ISSUES_SECTION>

<PREVIOUS_INPUT_SECTION>

VULNERABILITY REPORT (from Code Analyst):

<REPORT_JSON>

INSTRUCTIONS FOR C++ GENERATOR:

1. Write a complete, compilable C++17 program ('int main() {...}').
2. The program must print EXACTLY ONE valid test case to standard output ('std::cout').
3. VALIDITY-FIRST: the generated input MUST satisfy all format and validator constraints before you try to make it adversarial.
4. Focus on producing the input data structures matching the 'input_hypothesis' in the report.
5. If the previous attempt was rejected, fix those exact issues before changing anything else.
6. DO NOT use uninitialized variables or undefined behavior in your generator.
7. If you need randomness, you MAY use '<random>' ('std::mt19937'), but since this is the Semantic Generator, deterministic construction of the edge case is preferred when possible.

CRITICAL FORMATTING RULES:

1. Return ONLY the C++ code.
2. DO NOT wrap the code in markdown blocks (e.g., "cpp ...").
3. The very first line should be '#include <...>' or similar valid C++.

Write the C++ generator code now:

Hacker — semantic.checklist

You are repairing a C++ Semantic Generator after a failed attack-input attempt. Analyze the failure and produce a compact JSON checklist before patching the code.

PROBLEM DESCRIPTION:

<PROBLEM_DESC>

INPUT VALIDITY CONSTRAINTS:

<CONSTRAINTS_TEXT>

<ADVICE_SECTION>

LATEST FAILURE TYPE: <FAILURE_KIND>

LATEST FAILURE REASON:

<FAILURE_REASON>

<ISSUES_SECTION>

<INPUT_SECTION>

VULNERABILITY REPORT:

<REPORT_JSON>

PREVIOUS GENERATOR CODE:

<LAST_GENERATOR_CODE>

CHECKLIST RULES:

1. VALIDITY BEFORE ATTACK: first restore validator-accepted input, then preserve adversarial intent.
2. 'must_fix' must focus on the concrete compile/runtime/validator issue from the latest failure.
3. 'do_not_regress' must list already-required legality properties that must remain true.
4. 'attack_goal' must keep the attack aligned with the analyst report instead of collapsing into trivial valid input.
5. Be failure-type aware:
 - compile_failed -> prioritize syntax/API/build repairs
 - validator_rejected -> prioritize legality/format repairs
 - runtime_error -> prioritize execution safety
 - empty_output -> prioritize guaranteed emission of one valid case

Return ONLY valid JSON with exactly this schema:

```
{
  "must_fix": [...],
  "do_not_regress": [...],
  "attack_goal": [...]}
}
```

Hacker — semantic.patch

You are applying a minimal patch to a C++ Semantic Generator. You must patch the existing generator with minimal SEARCH/REPLACE edits instead of rewriting it from scratch.

PROBLEM DESCRIPTION:

<PROBLEM_DESC>

```

INPUT VALIDITY CONSTRAINTS:
<CONSTRAINTS_TEXT>
<ADVICE_SECTION>
LATEST FAILURE TYPE: <FAILURE_KIND>
LATEST FAILURE REASON:
<FAILURE_REASON>
<INPUT_SECTION>
VULNERABILITY REPORT:
<REPORT_JSON>

REPAIR CHECKLIST:
<CHECKLIST_JSON>

CURRENT GENERATOR CODE:
<LAST_GENERATOR_CODE>

PATCH RULES:
1. VALIDITY-FIRST: fix the latest failure before any attack refinement.
2. Preserve already-correct code whenever possible.
3. Keep the attack goal from the checklist after legality is restored.
4. Output only SEARCH/REPLACE blocks.
5. Each SEARCH block must match the current generator code exactly once.
6. Make the patch minimal and surgical. Do NOT replace the whole file.

Required patch format:
<PATCH_FORMAT>

```

Hacker — stress.generator

You are the Stress Test Generator, a specialized C++ coder for an adversarial Hacker System. Your job is to write a standalone C++ program ('int main()') that acts as a high-throughput Fuzzer.

PROBLEM DESCRIPTION:
<PROBLEM_DESC>

CONSTRAINTS (The output of your C++ fuzzer MUST strictly satisfy these boundaries):
<CONSTRAINTS_TEXT>

INSTRUCTIONS FOR C++ FUZZER:

1. Write a complete, compilable C++17 program.
2. VALIDITY-FIRST: the generated input MUST pass the problem validator and explicitly enforce all structural constraints in code.
3. The program must print EXACTLY ONE valid test case to standard output, but this test case should be as LARGE and COMPLEX as the constraints allow.
4. You MUST use '<random>' and 'std::mt19937_64' initialized with a random device or fixed seed.
5. Scale up the generation loop to approach the maximum 'N', 'M', or 'K' allowed.
6. Emphasize boundary values (e.g. generating values alternating between min and max allowed).
7. Optimize the generator for speed using '\n' instead of 'std::endl' and fast I/O ('std::ios_base::sync_with_stdio(false)').

CRITICAL FORMATTING RULES:

1. Return ONLY the C++ code.
2. DO NOT wrap the code in markdown blocks (e.g., "cpp ...").
3. The very first line should be '#include <...>' or similar valid C++.

Write the C++ Stress Test Generator code now:

Hacker — stress.checklist

You are repairing a C++ Stress Test Generator after a failed attempt. Analyze the failure and produce a compact JSON checklist before patching the code.

PROBLEM DESCRIPTION:
<PROBLEM_DESC>

INPUT VALIDITY CONSTRAINTS:
<CONSTRAINTS_TEXT>

LATEST FAILURE TYPE: <FAILURE_KIND>
LATEST FAILURE REASON:
<FAILURE_REASON>
<ISSUES_SECTION>
<INPUT_SECTION>
PREVIOUS GENERATOR CODE:
<LAST_GENERATOR_CODE>

CHECKLIST RULES:

1. VALIDITY BEFORE ATTACK: first restore validator-accepted input, then keep the case large and boundary-heavy.
2. 'must_fix' must target the concrete compile/runtime/validator failure from the latest attempt.
3. 'do_not_regress' must preserve constraints already satisfied by the generator.
4. 'attack_goal' must keep the output large, high-throughput, and stress-oriented after repairs.
5. Be failure-type aware:

- compile_failed -> prioritize syntax/API/build repairs
- validator_rejected -> prioritize legality/format repairs
- runtime_error -> prioritize execution safety
- empty_output -> prioritize guaranteed emission of one valid case

Return ONLY valid JSON with exactly this schema:

```
{
  "must_fix": ["..."],
  "do_not_regress": ["..."],
  "attack_goal": ["..."]
}
```

Hacker — stress.patch

You are applying a minimal patch to a C++ Stress Test Generator.
You must patch the existing generator with minimal SEARCH/REPLACE edits instead of rewriting it from scratch.

PROBLEM DESCRIPTION:
<PROBLEM_DESC>

INPUT VALIDITY CONSTRAINTS:
<CONSTRAINTS_TEXT>

LATEST FAILURE TYPE: <FAILURE_KIND>
LATEST FAILURE REASON:
<FAILURE_REASON>
<INPUT_SECTION>
REPAIR CHECKLIST:
<CHECKLIST_JSON>

CURRENT GENERATOR CODE:
<LAST_GENERATOR_CODE>

PATCH RULES:

1. VALIDITY-FIRST: fix the latest failure before increasing attack pressure.
2. Preserve already-correct large-case generation logic when possible.
3. Keep the generator large, randomized, and boundary-oriented after the repair.
4. Output only SEARCH/REPLACE blocks.
5. Each SEARCH block must match the current generator code exactly once.
6. Make the patch minimal and surgical. Do NOT replace the whole file.

Required patch format:
<PATCH_FORMAT>

Hacker — anti_hash.generator

You are the Anti-Hash Generator, the cryptography/math specialist for an adversarial Hacker System.
Your job is to read the Code Analyst's report identifying a polynomial rolling hash susceptibility, and write a standalone C++ program to generate colliding strings.

PROBLEM DESCRIPTION:
<PROBLEM_DESC>

CONSTRAINTS (Output MUST SATISFY ALL string/length constraints):
<CONSTRAINTS_TEXT>

VULNERABILITY REPORT (from Code Analyst):
<REPORT_JSON>

INSTRUCTIONS FOR C++ GENERATOR:

1. Write a complete, compilable C++17 program ('int main()').
2. VALIDITY-FIRST: the generated collision input MUST pass the problem validator and satisfy all structural constraints.
3. Implement a collision derivation algorithm (e.g., Thue-Morse sequence, Birthday attack, or specific moduli exploitation) matching the 'input_hypothesis'.
4. Output EXACTLY the string(s) needed to trigger the collision to standard out.
5. Keep the target constraints in mind. If max length is 'N=10^5', the collision strings must not exceed this length.

CRITICAL FORMATTING RULES:

1. Return ONLY the C++ code.
2. DO NOT wrap the code in markdown blocks (e.g., "cpp ...").
3. The very first line should be '#include <...>' or similar valid C++.

Write the C++ Collision Generator code now:

When a generator's output is rejected, a per-route *checklist* prompt produces a JSON repair plan (*must_fix*, *do_not_regress*, *attack_goal*); a per-route *patch* prompt then applies minimal SEARCH/REPLACE edits with the same format as the Solver patch prompt. The full reward shaping for the Hacker (severity weights, valid/break ratios, generator-failure penalty) is given in Appendix D.

F Experimental Configuration

This appendix collects the exact backbone, infrastructure, and budget settings used in all reported experiments.

F.1 Backbones and inference

All five backbones are accessed through a unified Azure OpenAI-compatible gateway with AAD authentication. Within each row of the main comparison and each reported ablation subset, every method is driven by the same backbone deployment so that comparisons isolate the agent framework rather than the underlying model.

Backbone deployments and decoding

Models used:

| | |
|-----------------|---|
| GPT-5.4 | gpt-5.4 (default and per-role for code/generator/validator/checker/output/hacker) |
| Claude Opus 4.6 | claude-opus-4-6 |
| Qwen3.6 | local serving via OpenAI-compatible endpoint |
| DeepSeek V4 Pro | deepseek-v4-pro |
| Grok | grok-4 |

Decoding: temperature = 0.1, max_tokens = 16,384 (default; raised to 64,000 for long-context backbones).

Embeddings: runtime retrieval uses text-embedding-3-small via the same gateway, with a 32,768-entry LRU cache and up to 5 HTTP retries; offline corpus deduplication uses text-embedding-3-large as described in Section 2.3; both fall back to local Sentence-Transformers when the gateway is unavailable.

API: OpenAI-compatible gateway; provider-specific endpoint and authentication details are released with the public code.

F.2 Pipeline budgets

The closed-loop pipeline (Section 3.1) runs under fixed iteration budgets to make per-problem cost comparable across baselines.

Per-problem pipeline budgets

Solver inner loop: max_iterations = 8 patch/regenerate rounds before the candidate is declared failed.

Hacker outer loop: max_hack_rounds = 3 rounds; each round runs the full Code-Analyst → generator → validator cascade.

Oracle certification: target N_{target} certified tests per problem; acceptance gate $\rho \geq \tau$ (Eq. 3). On rejection the Oracle retries with a different solver family selected by the bandit.

Token cap: 16,384 tokens per LLM call (matched across all baselines we report).

F.3 Knowledge-network defaults

Each agent’s knowledge network is a contextual-bandit policy (Appendix B) over a typed item store. The Solver additionally carries the Solver knowledge network (Section 3.3). Default retrieval and sampling budgets:

Solver QMS network (config/solver_network.yaml)

enabled: true, graph_dir: artifacts/solver_network/latest/graph
Top-k similar Q nodes per query: 4
Skill candidate pool: top 20, sampled 5 at skill_selection_temperature = 0.2
Skill bounds enforced by LLM selection prompt: min_llm_skills = 1, max_llm_skills = 5
Augmentation block: skill text only by default; full templates injected only when include_skill_templates_in_augmentation is on.
Optional ensemble mode runs 3 independent skill-plan + CodeGen/Hacker tails in parallel after Test Generation, then merges the best branch.

Trainable bandit memory (config/trainable_memory.yaml)

enabled: true, data_dir: artifacts/trainable_memory
Per-namespace top-k injected as advice: plan_top_k = 3, solve_top_k = 3, test_top_k = 3, hack_top_k = 3, oracle_top_k = 3.
Bandit learning rate $\alpha = 0.01$; tag-overlap prior +0.05 per matching tag.
Items with average reward < -0.3 after ≥ 20 uses are auto-deprecated.
Persistence: SQLite-backed JSON store with atomic file-locking writes.

F.4 Datasets and benchmarks

Benchmarks used in Tables 1–2

CodeContests (CC): 165 problems sampled from the DeepMind CodeContests test split.
APPS: 1,000 problems sampled across the introductory / interview / competition tiers.
AetherCode (AC): 400 held-out problems combining novel algorithmic tasks with verified Oracle-generated test suites.
Codeforces (Fig. 6b): recent Div. 2 and Div. 1+2 rounds, attempted under the official time limit in a single uninterrupted session per round.
Cold-start corpus: 8,017 final problems scraped directly from Codeforces, AtCoder, Aizu Online Judge, and a long tail of smaller platforms (LeetCode, SPOJ, UOJ, etc.), obtained from 30,018 raw problems after the four-stage filtering pipeline of Section 2.
Primary metric: pass@1 across all four benchmarks.

F.5 Codeforces Rating Estimation Protocol

This appendix gives the exact procedure behind the Codeforces-style rating curves in Figure 6b. The goal is not to claim an official Codeforces account rating, which depends on platform-side online updates and eligibility rules, but to report a human-comparable contest-local rating estimate from the same public ingredients used in Codeforces standings: official accepted submissions, solved counts, penalties, ranks, and pre-contest human ratings. We follow the rating-inversion view used by CodeElo [15], which is itself based on the Elo expected-score model [16].

Contest protocol. Let $\mathcal{C} = \{c_1, \dots, c_K\}$ be the ordered set of recent Codeforces rounds used in Figure 6b. Each $c \in \mathcal{C}$ is attempted under its official duration and division setting (Div. 2 or Div. 1+2). For each agent/backbone pair a , Solvita starts from the contest statements at time zero and runs one uninterrupted session. No manual corrections, prompt edits, or submissions after the official contest window are allowed. A problem is counted as solved only if the official judge returns Accepted before the contest closes; pretests-only success, wrong answer, time limit exceeded, memory limit exceeded, runtime error, compilation error, and post-window acceptance all count as unsolved for that contest.

The Codeforces runs use the fixed contest-time configuration in Table 10. These budgets match Appendix F unless explicitly listed.

Table 10: Contest-time configuration used for the Codeforces rating-estimation runs.

| Setting | Value | Role in the evaluation |
|-----------------------------|--|--|
| Decoding temperature | 0.1 | Low-variance generation for both Solvita and bare-backbone baselines. |
| Per-call token cap | 16,384 | Maximum output budget for each LLM call, matched across compared systems. |
| Contest window | Official duration of each round | All submissions must be made before the contest closes; post-window acceptance is counted as unsolved. |
| Solver inner-loop budget | max_iterations = 8 | Maximum patch/regenerate attempts before a candidate is declared failed. |
| Hacker outer-loop budget | max_hack_rounds = 3 | Number of adversarial test-generation rounds per candidate. |
| Solver skill candidate pool | top 20 retrieved, 5 sampled | Candidate skills retrieved from the Solver QMS network before LLM selection. |
| Skill sampling temperature | skill_selection_temperature = 0.2 | Controls stochasticity when sampling skill candidates. |
| LLM-selected skill bounds | min_llm_skills = 1, max_llm_skills = 5 | Lower and upper bounds on the number of skills injected into Solver context. |
| Bare-backbone ablation | Solvita loop disabled | Same deployment, decoding temperature, token cap, contest window, and official-judge scoring, but without Planner/Solver/Oracle/Hacker coordination or trainable knowledge networks. |

Standings insertion. For a contest c , let H_c denote the set of official human participants retained for rating estimation after removing unrated or missing-rating entries. Each participant $i \in H_c$ has a pre-contest Codeforces rating $R_{c,i}$ and an official standing tuple

$$z_{c,i} = (S_{c,i}, -P_{c,i}, -L_{c,i}), \quad (7)$$

where $S_{c,i}$ is the number of solved problems, $P_{c,i}$ is the total Codeforces penalty, and $L_{c,i}$ is the last accepted-submission time used only as a deterministic tie-breaker when necessary. For an agent a , we construct the analogous tuple

$$z_{a,c} = (S_{a,c}, -P_{a,c}, -L_{a,c}) \quad (8)$$

from its official submissions during the contest window. The agent’s inserted rank is then

$$m_{a,c} = 1 + \sum_{i \in H_c} \mathbb{1}[z_{c,i} \succ z_{a,c}], \quad (9)$$

where $\succ$ is the official standing order: more solves rank higher, lower penalty ranks higher among equal solves, and the final tie-breaker is applied only to make the rank deterministic. Thus $m_{a,c} = 1$ means the agent is inserted above every retained human participant, and $m_{a,c} = |H_c| + 1$ means it is below all retained human participants.

Contest-local rating inversion. Let r be the latent rating whose expected number of human participants outperforming the agent matches its inserted rank. Under the Elo model, the probability that human participant i outperforms a player of rating r is

$$p_{i \succ r} = \frac{1}{1 + 10^{(r - R_{c,i})/400}}. \quad (10)$$

We therefore solve for the contest-local rating estimate $\hat{r}_{a,c}$ satisfying

$$m_{a,c} - 1 = \sum_{i \in H_c} \frac{1}{1 + 10^{(\hat{r}_{a,c} - R_{c,i})/400}}. \quad (11)$$

The right-hand side is strictly decreasing in $\hat{r}_{a,c}$, so the solution is unique up to ties in the empirical standings. In implementation we use binary search over a wide Codeforces-compatible interval, e.g. $[-500, 5000]$, until the residual in Eq. 11 is below 10^{-6} :

$$\hat{r}_{a,c} = \arg \min_r \left| m_{a,c} - 1 - \sum_{i \in H_c} \frac{1}{1 + 10^{(r - R_{c,i})/400}} \right|. \quad (12)$$

This inversion differs from the online Codeforces update rule in that each contest is treated independently; it is appropriate here because all compared agents participate in the same fixed contest set and we want a low-variance, contest-local estimate rather than a platform account history.

Aggregation into Figure 6b. For each agent a and ordered contest prefix $\{c_1, \dots, c_t\}$, the plotted rating trajectory is the running mean of contest-local estimates:

$$\bar{r}_{a,t} = \frac{1}{t} \sum_{\ell=1}^t \hat{r}_{a,c_\ell}. \quad (13)$$

The final estimate after all K contests is

$$\bar{r}_a = \bar{r}_{a,K} = \frac{1}{K} \sum_{c \in \mathcal{C}} \hat{r}_{a,c}. \quad (14)$$

When reporting uncertainty, we use the across-contest standard error

$$\text{SE}(\bar{r}_a) = \sqrt{\frac{1}{K(K-1)} \sum_{c \in \mathcal{C}} (\hat{r}_{a,c} - \bar{r}_a)^2}. \quad (15)$$

The tier bands in Figure 6b are the canonical Codeforces color/rating bands applied to $\bar{r}_{a,t}$ only for interpretability; they do not imply that the evaluated agents are official Codeforces users.

G Knowledge-Network and Pipeline Implementation Details

This appendix documents implementation details of the four knowledge networks and the closed-loop control flow that sit behind Section 3.

G.1 Shared item schema

All four namespaces (Plan, Solve, Test/Oracle, Hack) share an SQLite-backed item table. Each entry stores: a stable id (primary key), a namespace tag, a human-readable summary, a role-specific JSON payload, a list of searchable tags, a usage counter, a running average reward, a deprecation flag, and creation/last-used timestamps. Reads are served from an in-memory index; writes go through atomic file-locked transactions so that concurrent benchmark workers do not corrupt the store.

G.2 Featurizer and bandit scoring

At inference time, a per-namespace featurizer maps the current pipeline state to a sparse set of feature keys $\Phi(x)$. The keys cover three orthogonal axes:

- **FSM position:** FSM:SOLVE_DRAFT, FSM:SOLVE_PATCH, FSM:HACK_SEMANTIC, etc.
- **Failure type from the previous iteration:** FAIL:WA, FAIL:TLE, FAIL:RE, FAIL:MLE, FAIL:OFF_BY_ONE, ...
- **Problem-level tags:** TAG:dp, TAG:graphs, TAG:strings, ...

Each item i is scored as $\text{score}(i) = b_i + \sum_{f \in \Phi(x)} W_{f,i}$, with a +0.05 bonus per matching tag. The top- k items (typically $k = 3$, see Appendix F) are selected via ϵ -greedy exploration and injected into the prompt as the <MEMORY_ADVICE> placeholder.

G.3 Solver knowledge network storage and dynamics

The Solver knowledge network is persisted under `artifacts/solver_network/latest/graph` as a node-edge bundle plus a checkpoint of the learned w_{qm} and w_{ms} matrices. Each Q node stores the canonical problem JSON and tag list; each M node stores either an analysis trajectory (single solution decomposed into a function-block DAG) or a contrastive pair (correct + incorrect solution sharing the same approach, with the divergence point annotated); each S node stores an annotated skill with a C++ template and usage notes. Q-M edges are created at corpus-build time from authored or extracted decompositions; M-S edges use either deterministic function-block identifier matches (preferred) or an embedding-similarity fallback when no deterministic match exists. Edge weights are renormalized within each MS group after every REINFORCE update so that $\sum_s w_{ms}^{(j,s)} = 1$.

G.4 Failure-event propagation

When the Hacker breaks a candidate, the failure event is broadcast to all four namespaces with role-specific payloads: the Plan namespace records the failed paradigm (negative reward on the strategy item), the Solve namespace creates a new contrastive M node pairing the broken solution with the closest correct sibling, the Oracle namespace records the missed input pattern as a generator hint, and the Hack namespace bumps the successful route’s score. This single mechanism is what makes one discovered bug propagate as a lesson across the entire pipeline rather than being lost after the round.

G.5 Sandbox and judge resolution

All compilation and execution happens in a per-process sandbox (`src/sandbox/`) that wraps `g++ -std=c++17 -O2` with wall-time and memory limits. The judge is resolved in a strict priority order, identical for the Oracle certification gate and the Hacker break check: (1) a custom checker if one exists, (2) running the certified reference solution and comparing tokens, (3) exact-match against the canned expected output. This priority defines the per-test verdicts that feed both the Oracle certification ratio $\rho(x, f)$ in Eq. 5 and the Hacker break-rate g_{break} in Eq. 4.

H Additional Ablations

This appendix expands on the secondary ablations referenced from the experiments section. The main paper reports the headline numbers in Table 3 (RQ3, patch vs. regenerate) and Figure 6a (RQ4, Oracle/Hacker decomposition); here we describe the protocol and the additional sweeps we ran.

Diagnostic metrics for Figure 6a. Figure 6a evaluates each diagnostic configuration against held-out candidate solutions with known official verdicts. Solvita is said to *accept* a candidate if the diagnostic procedure does not find a failure, and to *reject* it if the procedure flags the candidate as incorrect. For candidate-level accounting, we use the following four outcome counts:

| | Official correct | Official wrong |
|-----------------|------------------|----------------|
| Solvita accepts | TP | TN |
| Solvita rejects | NP | NF |

Here, TP denotes candidates accepted by both Solvita and the official judge; TN denotes candidates accepted by Solvita but rejected by the official judge; NP denotes candidates rejected by Solvita but accepted by the official judge; and NF denotes candidates rejected by both Solvita and the official judge.

The wrong-solution detection rate, reported as Det'd, measures the fraction of officially wrong candidates that Solvita detects:

$$\text{Det'd} = \frac{NF}{TN + NF} \times 100\%.$$

The correct-solution preservation rate, reported as Pres'd, measures the fraction of officially correct candidates that Solvita does not reject:

$$\text{Pres'd} = \frac{TP}{TP + NP} \times 100\%.$$

The third metric, reported as Str. Rate, is computed only over Solvita-rejects/official-accepts disagreements and is aggregated at the problem level rather than the candidate level. These disagreements correspond to the NP cell above, but we do not automatically treat them as Solvita errors: in some cases Solvita may have generated stronger diagnostic tests that expose hidden failures in solutions accepted by the official test suite. Let $\mathcal{D}_{NP}$ be the set of problems for which Solvita rejects at least one officially accepted candidate. For each problem $d \in \mathcal{D}_{NP}$, we collect a pool $\mathcal{A}_d$ of standard accepted candidate solutions and run every solution in $\mathcal{A}_d$ against Solvita's diagnostic tests for d . Let $\mathcal{B}_d \subseteq \mathcal{A}_d$ be the subset of these accepted candidates that are rejected by Solvita's diagnostic tests:

$$\mathcal{B}_d = \{a \in \mathcal{A}_d : a \text{ fails Solvita's diagnostic tests for } d\}.$$

A disagreement problem d is counted as a confirmed stronger-test case only when both of the following conditions hold:

$$\frac{|\mathcal{B}_d|}{|\mathcal{A}_d|} > 0.10,$$

and manual inspection confirms that the failing diagnostic examples have valid inputs under the problem statement and constraints, with correct expected outputs or correct checker verdicts. The first condition rules out isolated failures of a single accepted candidate; the second condition rules out invalid inputs, incorrect expected outputs, checker mistakes, and Oracle/certification errors. Let $\mathcal{D}_{\text{strong}} \subseteq \mathcal{D}_{NP}$ be the set of disagreement problems satisfying both conditions. We define the confirmed stronger-test rate as

$$\text{Str. Rate} = \frac{|\mathcal{D}_{\text{strong}}|}{|\mathcal{D}_{NP}|} \times 100\%.$$

Thus, Str. Rate measures the share of Solvita-rejects/official-accepts disagreement problems that are confirmed, through accepted-solution cross-checking and manual validation, to reflect genuinely stronger diagnostic tests rather than unsupported Solvita rejections.

Patch repair vs. full regeneration. We rerun the Solver inner loop with two settings holding everything else fixed: (i) *patch*, the default, which routes through `generate_code.patch_decision` and emits SEARCH/REPLACE blocks (Appendix E); (ii) *regenerate*, which always falls through to `generate_code.regenerate` and rewrites the full program. Both modes share the same iteration cap ($N_{\text{max}} = 8$) used end-to-end (Appendix F) and the same failure-analysis prompt. We additionally measure the regression rate (fraction of iterations that break a previously passing test), which is the mechanism Section 4.3 attributes the gain to.

LLM skill selection vs. pure softmax sampling. The default Solver uses an LLM skill-selection step (Appendix E, `solver_skill_selection`) on top of the softmax over $\rho(s \mid q_{\text{new}})$ from Eq. 1. We ablate by replacing the LLM step with direct top- k sampling from $\pi(s)$. The LLM step adds a non-trivial cost but consistently selects more coherent skill bundles (matched to the sub-problem DAG it co-emits), and the gap widens on problems whose paradigm class is heterogeneous in the QMS retrieval set.

Oracle acceptance threshold τ sensitivity. The Oracle gate (Eq. 3) accepts an artifact only when the certification ratio $\rho \geq \tau$. We sweep $\tau \in \{0.6, 0.75, 0.9, 1.0\}$. Lower τ admits more tests but changes the downstream detection-preservation trade-off in Figure 6a; the default $\tau = 0.9$ is the knee of the precision/recall trade-off.

Hacker round budget. `max_hack_rounds` is set to 3 by default (Appendix F). Sweeping in $\{1, 2, 3, 5\}$ shows that most break events occur in rounds 1–2 (semantic + stress) and the marginal gain from round 3 (typically anti-hash on hashed problems) is small but non-zero; rounds 4+ rarely surface new bugs. We therefore set the default to 3 rather than 2 so that the anti-hash route gets a fair turn on hash-based problems where it is the only effective attack, while still avoiding the wasted budget at rounds 4+.

Contrastive vs. non-contrastive REINFORCE. The Solver REINFORCE update (Eq. 2) uses a counterfactual reward $\Delta R = R_{\text{with}} - R_{\text{without}}$ from a paired with/without-network rollout. Replacing ΔR with the absolute reward R_{with} removes the variance-reduction baseline and slows convergence across all three checkpoints of Tab. 2; the contrastive form is what makes the Solver knowledge network the dominant component in Tab. 2.

I Failure Cases

We document representative failure modes observed during evaluation. Each is a real category we encountered and that informed the design choices documented in Sections 3.5–3.6 and the prompts in Appendix E.

Cold-start retrieval misfires. On problems whose paradigm is poorly represented in the cold-start corpus, the Solver QMS network retrieves structurally similar but semantically misleading skills. The selection LLM (Appendix E, `solver_skill_selection`) is permitted to return an empty list in this case, but it does not always do so, and the resulting skill block can bias the initial draft toward the wrong paradigm. The mitigation is the contrastive REINFORCE signal of Eq. 2: paired rollouts attribute negative reward to the misleading skill node and downweight it within a few episodes, but the first few problems of each new paradigm class are over-represented in the failure set.

Oracle false certification. Despite the multi-stage gate (Eq. 3), the Oracle occasionally accepts a reference solution that contains a subtle bug agreeing with the canonical inputs. When this happens, certified tests inherit the bug and the Solver is rewarded for matching it. The Hacker’s adversarial round catches most such cases (semantic route on edge cases the Oracle did not enumerate), but pathological agreements between two independently buggy implementations remain a residual failure mode and motivate the priority order *custom checker* > *correct-solution runner* > *exact match* described in Appendix G.

Hacker scope limitations. Problems that hinge on deep mathematical reasoning (number theory, combinatorial identities) frequently survive all three Hacker routes not because the candidate is correct but because the Code Analyst lacks the model capacity to identify the bug class. This is visible in the Hacker reward distribution as a bimodal pattern: high break-ratio on implementation-level bugs, near-zero on math-heavy problems. We currently do not have a systematic mitigation; warm-starting the Hack network from human-authored counterexamples on a held-out math subset is the most promising next step.

Patch repair drift on global flaws. The `patch_decision` prompt (Appendix E) is meant to route systemic flaws to `full_regen`, but on borderline cases it can mislabel a global flaw as localized and start patching. The result is a sequence of small edits that each fix the immediate failure but accumulate state inconsistencies, eventually exhausting `max_iterations` (Appendix F) without converging. We currently rely on the regression-rate signal in Section 4.3 to identify these runs after the fact; integrating that signal into `patch_decision` as a feature is straightforward future work.